The National University of Computer and Emerging Sciences, Lahore

Course Notes

Compiler Construction

CS-4031

Semester Fall-2026

Compiled lecture notes – see individual lectures for per-lecture references.

Contents

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 1

How Programs Run: Compilers, Interpreters, and JIT Compilation (a Java Case Study); the Language-Processing System; Why Study Compilers; Kinds of Compilers

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.1 (Language Processors), §1.2 (Language-Processing System – previewed here, detailed in Lecture 2), §1.4 (Compiler/Interpreter Distinction), §1.5 (Applications of Compiler Technology)
Other references:	[ET] Engineering a Compiler §1.1; [AP] Modern Compiler Implementation, Ch. 1

Here is what today actually covers:

Lecture Roadmap

1. What is a language processor, and what exactly is a *compiler*?
2. Compilers vs. interpreters vs. just-in-time (JIT) compilation — three ways to run a program, with a full case study of how **Java** uses this to become portable.
3. A **typical language-processing system**: preprocessor, compiler, assembler, linker, and loader – and exactly what an **object file** is.
4. Why study compilers at all, even if you never write one professionally?
5. Applications of compiler technology far beyond "turning C into an executable."
6. Six different **kinds of compiler** you'll encounter by name in industry.

This lecture is entirely conceptual — no algorithms yet. Lecture 2 goes much deeper into the internal *structure* (phases) of a compiler.

1.1 What Is a Language Processor?

[DB] A **language processor** is any program that takes another program written in some source notation and does something useful with it — runs it, translates it, checks it, or transforms it. Compilers are the most famous member of this family, but not the only one.

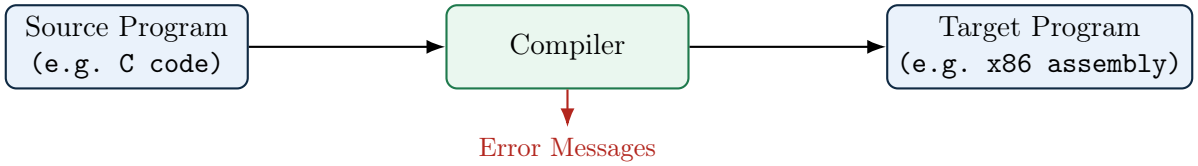
Definition: Compiler

A **compiler** is a program that reads a program written in one language — the *source language* — and translates it into an equivalent program in another language — the *target language*. Along the way, if the source program has errors, the compiler should report them to the user in a useful way.

Three things are packed into that definition and worth pulling apart, because exam questions love to test exactly this distinction:

- **Translation, not execution.** A compiler's job is to *produce* a target program. It does not itself run your code. If you compile `main.c` with GCC, GCC's job ends when it hands you `a.out`; a completely different program (the operating system's loader, then the CPU) is what actually executes it.

- **Equivalence.** The target program must have the same meaning as the source program. This is the single hardest engineering promise a compiler makes, and essentially the entire field of compiler optimization exists to preserve this promise while still making the target program faster.
- **The target language need not be machine code.** Nothing in the definition says what the target language *is* – only that it is "another language." Machine code is the single most common and most visible target, which is why people default to thinking "compiler = produces machine code," but it is only one case. A compiler whose target is *another high-level language* is a source-to-source compiler (§1.6); a compiler whose target is bytecode for a virtual machine (`javac` → JVM bytecode, §1.2.5) is still a compiler by this same definition, even though no physical CPU can execute its output directly.



Exam Focus

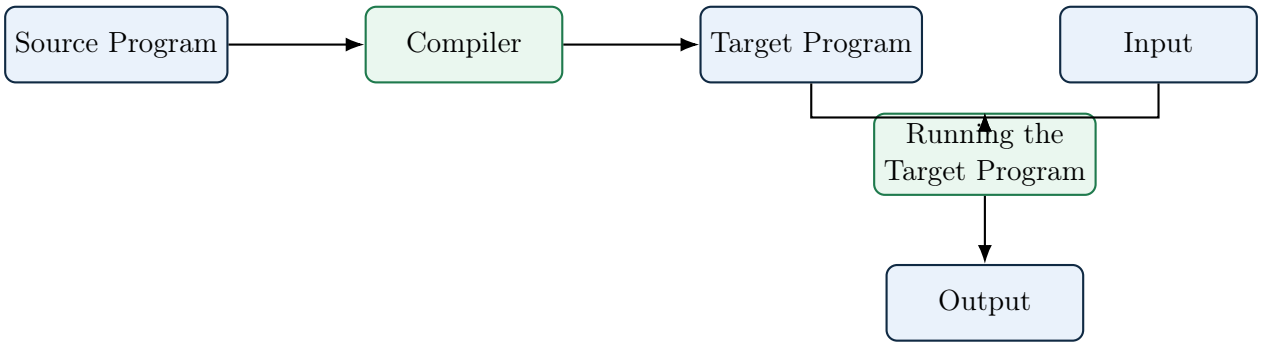
Be able to state, in one sentence, that a compiler *translates*; it does not *execute*. A one-line definition question like "define a compiler" is a classic first-quiz question and students routinely lose marks by describing what an interpreter does instead – or by adding "... into machine code" to the definition, which is not required and is contradicted by transpilers and by `javac` (§1.6).

1.2 Compilers vs. Interpreters vs. JIT Compilation

This is the heart of today's lecture (DB §1.4). There are three broad strategies for getting from source code to a running program, and almost every real language-processing system you use day to day is one of these three, or (very commonly) a hybrid.

1.2.1 Pure Compilation (Ahead-of-Time)

[DB] The compiler translates the entire source program into a target program *before* execution begins, and the target program can then be run any number of times, independently of the compiler. In the classic AOT case that gives this strategy its name, the target language is machine code for the host machine – this is what C, C++, Rust, and (classic) Fortran compilers produce – but the "ahead-of-time, whole-program-at-once" strategy itself is independent of that choice of target (a transpiler, §1.6, is also AOT; it just targets another high-level language instead of machine code).



Advantage – maximal optimization opportunity. Because the entire source program is available before a single instruction runs, an AOT compiler can afford analyses and transformations that a system under run-time time pressure cannot: multiple, deliberately ordered optimization

passes; *whole-program* (interprocedural) analysis such as global register allocation or cross-function inlining; and expensive, exhaustive search over code-generation choices. None of this has to finish before the user sees output, since compilation happens entirely offline – only the resulting target program's run time is on the user's clock.

Disadvantage – the output is not portable. The target program an AOT compiler produces is machine code for one specific instruction set, calling convention, and (usually) operating system. A binary built for x86-64 Linux will not run on ARM64 macOS: there is no platform-independent artifact to ship. Reaching a new target means recompiling from source for that target (or cross-compiling, §1.6) – unlike an interpreter, which ships the same source to every platform, or a bytecode/JIT system, which ships the same portable bytecode everywhere.

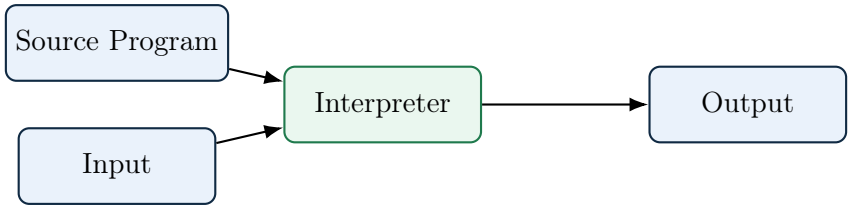
Trade-off, restated: compilation itself can be slow and happens "offline," but every run of the target program afterward is fast, because all the translation work – and all the optimization it enables – has already been paid for once, at the cost of tying the result to one platform.

1.2.2 Pure Interpretation

[DB] An **interpreter** does not produce a separate target program at all. Instead, it directly executes the operations specified in the source program, using the source program (or a close representation of it) as input on every single run.

Definition: Interpreter

An interpreter is a language processor that reads a source program and, using the input supplied by the user, *directly* produces the output of that program — with no separate translation phase that produces standalone target code the user can run independently.



Classic (older) Python, shell scripts, and simple Basic interpreters are close to this pure model: the interpreter re-examines and re-dispatches on the source (or its parsed form) every single time a statement executes, even inside a loop that runs a million times.

Advantage – portability. The same source program (or bytecode) runs unmodified on any machine that has a compatible interpreter, with no separate build step per target platform. This is the mirror image of the AOT compiler's disadvantage above.

Disadvantage – repeated-work overhead. There is no separate compile step, so you can start running – and get feedback on – code instantly, which is wonderful for interactive development and REPLs. The cost is that the interpreter re-examines and re-dispatches on the source (or its parsed form) on *every* execution of a line, with no persistent, whole-program analysis carried over between runs – the opposite of an AOT compiler's one-time, multi-pass optimization. DB notes this repeated-work overhead can make interpretation **10–100× slower** than running compiled code.

Exam Focus

Know this speed trade-off in both directions: compilation trades slower "build time" for faster "run time"; interpretation trades away build time for slower, but more flexible and immediate, run time. Two further pairings are just as commonly tested: *portability* is a classic interpreter advantage over AOT compilation (the same source runs unmodified on any machine that has the interpreter, whereas an AOT binary is tied to one target and needs a recompile elsewhere), while *optimization opportunity* runs the other way – an AOT compiler sees the whole program before

run time and can afford multiple, expensive, whole-program passes, while a pure interpreter re-examines the source on every execution and so cannot afford to.

1.2.3 The Hybrid Reality: Bytecode + JIT

[beyond DB] In practice, almost no serious modern language is *purely* compiled or *purely* interpreted. DB §1.4 briefly mentions Java's model as a hybrid; it is worth understanding this hybrid model properly, because it is how the overwhelming majority of code you run today — Java, C#, JavaScript, modern Python (via PyPy), and even Python's own CPython — actually executes.

Definition: Virtual Machine (VM), vs. a Plain Interpreter

A **virtual machine**, in the sense used here, is a program that simulates a computer: it defines its own instruction set (an abstract, software-only ISA) together with the state a real CPU would have – a program counter, and either an operand stack (JVM, Wasm) or a set of virtual registers – and then runs a fetch-decode-execute loop over that instruction set, exactly the way real hardware executes machine code, except every step is simulated in software. Crucially, a VM does *not* interpret your source program directly. There is always a separate front-end compilation step first (described just below) that translates source into **bytecode** – instructions for this abstract machine – and the VM interprets *that*, not your original code.
This is different from a "plain" (tree-walking) interpreter, of the kind §1.2.2 first introduced: a tree-walking interpreter parses source into an AST and directly, recursively evaluates that tree (`evaluate(node)` calling `evaluate(node.left)`, and so on). There is no separate instruction set, no fixed opcode encoding, and no simulated CPU state – the evaluator's own call stack mirrors the shape of the *source program*, not the shape of a machine. Nothing about it resembles hardware, so it does not earn the name "virtual machine."
The distinction, side by side:

- **What is executed.** VM: a fixed, formally-defined bytecode instruction set, produced by a separate compile step. Tree-walking interpreter: the source program's own parse tree, walked directly, with no separate instruction set at all.
- **Execution state.** VM: an explicit program counter plus an operand stack or virtual registers – state modeled on real hardware. Tree-walker: just the recursive call stack of the `evaluate` function, shaped like the program's grammar, not like a CPU.
- **Is it a genuine compilation target?** VM: yes – bytecode is often documented and shared across multiple front-end languages (JVM bytecode is targeted by Java, Kotlin, Scala, and Clojure alike). Tree-walker: no – the AST is just this one interpreter's private internal representation of this one program.

Every VM is (or contains) some form of interpreter – it interprets bytecode instruction by instruction. But not every interpreter is a VM: a tree-walker interprets source structure directly and never defines a machine to simulate. (A second, unrelated sense of "virtual machine" – *system* VMs like VMware or QEMU, which virtualize an entire computer including its OS – is a different concept entirely and not what is meant here.)

1. **Source → bytecode.** A "front-end" compiler translates source code into a compact, machine-independent intermediate form called **bytecode** (Java's `.class` files hold JVM bytecode; Python's `.pyc` files hold Python bytecode).
2. **Bytecode → execution.** A **virtual machine (VM)** then runs this bytecode. It can do this two ways:
   - *Interpret* the bytecode directly, instruction by instruction (slow but starts instantly) – this is what most bytecode VMs do at first.
   - *Just-in-time (JIT) compile* hot pieces of bytecode — the loops and functions that run

over and over — into native machine code *while the program is running*, and cache that native code for reuse.

Definition: Just-In-Time (JIT) Compilation

JIT compilation defers translation to native machine code until run time, and does so selectively: only code that is actually hot (executed frequently) is compiled, while cold code may remain interpreted. This lets a system get the fast startup of an interpreter and the fast steady-state speed of a compiler.

Why "Warm-Up" Adds a Delay

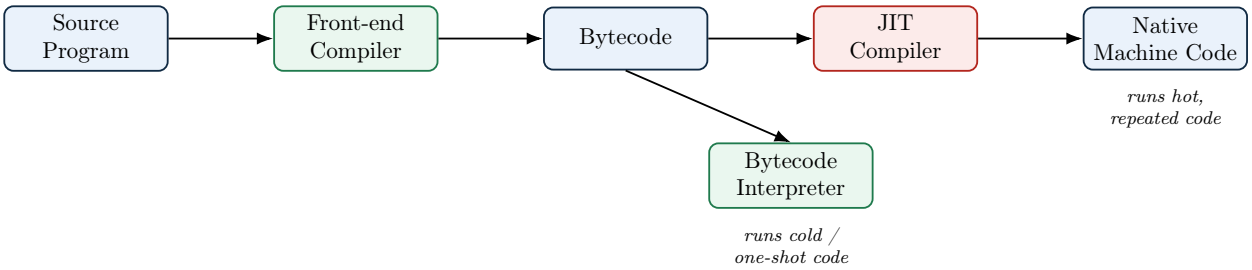
The "time to reach peak speed" row in §1.2.4's table (distinct from startup latency – see the exam-focus note there) has a concrete cause: three costs stack up, in sequence, before any piece of code reaches its fastest version, and none of them can be skipped.

1. **Early runs pay interpreter-speed cost.** The very first time a method or loop executes, no compiled native code for it exists yet – there is nothing to jump to – so it runs through the interpreter (or a cheap baseline tier, e.g. HotSpot's C1), which is inherently slower per operation than optimized native code.
2. **The system must first observe that code is hot.** A JIT does not compile on the first call; it counts. The runtime increments an invocation/back-edge counter on every call or loop iteration and only triggers compilation once that counter crosses a threshold. A loop that only runs a handful of times may never get compiled at all, because counting to that threshold, by construction, requires real executions to have already happened at the slower tier.
3. **Compilation itself takes real, non-trivial time.** Once flagged hot, the method still has to go through the JIT's own pipeline – build an IR from the bytecode, run optimization passes (inlining, register allocation, and more), and emit native code. This usually runs on a background thread so it does not block the program outright, but it still competes for CPU and memory (see "Extra runtime overhead" in §1.2.4's table), and the *old*, slower version keeps running until the newly compiled version is ready and swapped in.

The practical consequence: long-running processes (servers, long batch jobs) amortize this one-time warm-up cost over a long steady-state run and benefit enormously from JIT compilation, while short-lived scripts or CLI invocations may finish before ever reaching peak speed – exactly why AOT-vs.-JIT benchmarks can be misleading if they measure only a program that runs for a second or two.

Example: Tiered execution in real systems

- **JVM (HotSpot):** interprets bytecode first; the C1 ("client") JIT compiles warm methods quickly with light optimization; the C2 ("server") JIT recompiles the hottest methods with heavy optimization once enough profiling data exists.
- **V8 (Chrome/Node.js JavaScript engine):** `Ignition` interprets bytecode; `Sparkplug` does a fast baseline JIT; `TurboFan` does an aggressive optimizing JIT for hot functions, and can *deoptimize* back to the interpreter if an assumption used for optimization (e.g. "this variable is always a number") turns out to be wrong.
- **CPython (standard Python):** traditionally a pure bytecode interpreter with no JIT; `PyPy` is an alternative Python implementation that adds a JIT and is often 5–10× faster on long-running code as a result.



Extra Depth: AOT with a JIT Fallback

DB draws the compiler/interpreter line fairly simply. In practice, the line is blurrier still: some modern systems (GraalVM Native Image, .NET's ReadyToRun) precompile most code AOT for fast startup but still keep a JIT available for code discovered or specialized at run time. So "compiled" and "JIT-compiled" are not always mutually exclusive labels for one system. *(Transpilers, cross-compilers, bootstrapping compilers, and decompilers are their own category of "kind of compiler" rather than a compiler/interpreter distinction – we cover all of these properly in §1.6.)*

1.2.4 Comparing the Three Models

Table 1: Comparing the three execution models

	Pure Compiler	**Pure Interpreter**	**JIT / Hybrid**
When translation happens	Entirely before run time	Continuously, during run time	Source → bytecode happens before run time; bytecode → native code (the JIT step itself) is **always fully during** run time – see note below
Time to reach peak speed	N/A – already at peak from instruction one (see above)	N/A – there is no "peak"; every instruction runs at the same interpreted speed forever	**Nonzero ("warm-up")** – profiling must first identify hot code, then compilation must finish, before that code is upgraded to native speed; the program is already running the whole time this happens
Steady-state speed	Fastest	Slowest (re-dispatch overhead)	Approaches compiled speed for hot code
Portability	Target is machine-specific – a new platform needs a recompile (or cross-compile)	Source runs anywhere the interpreter exists	Bytecode is portable; the JIT-generated native code is not, but it never has to be shipped
Optimization opportunity	Largest – whole program visible before run time, so multiple, expensive, whole-program passes are affordable	Smallest – source (or bytecode) is re-examined on every execution, so heavyweight analysis is normally too costly to repeat	Bounded by run-time budget – fewer/cheaper passes than AOT and limited mostly to hot methods, but compensated by real profiling data (next row) unavailable to AOT

continued on next page

Table 1 – comparing the three execution models (continued)

	Pure Compiler	**Pure Interpreter**	**JIT / Hybrid**
Can use runtime profiling info?	No – only sees the source, never the running program	N/A – re-examines source every time, no persistent profile	**Yes** – observes actual branch frequencies and value types, enabling speculative optimizations no static compiler can safely make
Extra runtime overhead	None – compiler is long gone by run time	Dispatch overhead only	Profiling counters, background compiler threads, and a generated-code cache all compete with the program for CPU/memory
Debuggability / interactivity	Weakest (must recompile to test a change)	Best (REPL, immediate feedback)	Good (often still supports a REPL over bytecode)
Typical users	C, C++, Rust, Fortran	Shell scripts, classic BASIC	Java, C#, JavaScript, PyPy

Exam Focus

The "When translation happens" row is the row most often misread. The hybrid pipeline has *two* separate translation stages, and only one of them is "just-in-time":

- **Source → bytecode** (e.g. `javac` producing `.class` files) is ordinary ahead-of-time compilation. It happens once, before the program is ever run, exactly like §1.2.1's pure-compiler case – it is just that its *target* language is bytecode instead of machine code.
- **Bytecode → native code**, the actual JIT step, is *never* done ahead of time, not even partially. The decision of *which* loop or method is hot enough to be worth compiling is made by watching the program actually run (via profiling counters), and the native code generated is specific to the behavior observed *during that run* – this is exactly what "just-in-time" means: translation deferred until the last possible moment, triggered by run-time evidence, not decided in advance.

So the "partly before, partly during" language in the table describes the *full hybrid pipeline* (front-end compile, then JIT), not the JIT step in isolation. Taken alone, JIT compilation is entirely a run-time activity, no different in *when* it happens from pure interpretation – what makes it different is *what* it produces (a reusable native-code artifact) rather than *when* it produces it.

Exam Focus

A related trap, worth knowing even though the table above does not track it separately: "startup latency" and "warm-up delay" sound like the same thing but are not. **Startup latency** asks only: how long before the program starts executing at all? By that measure, JIT and pure interpretation are identical – a JIT-based system *begins* by interpreting (or running a cheap baseline tier) exactly like a pure interpreter does, so there is no extra pause before the first instruction runs. "Warm-up," by contrast, is not a delay *before* starting; it is the time *after* the program has already started before some of its code gets promoted to native speed – exactly what the "Time to reach peak speed" row above captures. The program is running the entire time warm-up is happening – it is simply running at interpreted (not yet peak) speed for that stretch. So it is correct to say a JIT system has low startup latency *and* a nonzero warm-up period – these are not in tension, because they answer two different questions.

A Bit of History

Interpretation is not a modern shortcut invented for scripting languages — it is *older* than compilation. Early LISP (1958) was interpreted because writing a LISP compiler was itself a research problem; interpreters let you experiment with a language before anyone knew how to compile it well. JIT compilation is also older than most people assume: the term traces to Smalltalk and self-modifying-code systems of the 1980s, and was popularized for the mainstream by Sun's HotSpot JVM in 1999 – it was Java's answer to "we want C-like speed but Smalltalk-like portability and safety."

1.2.5 Case Study: How Java Achieves Portability

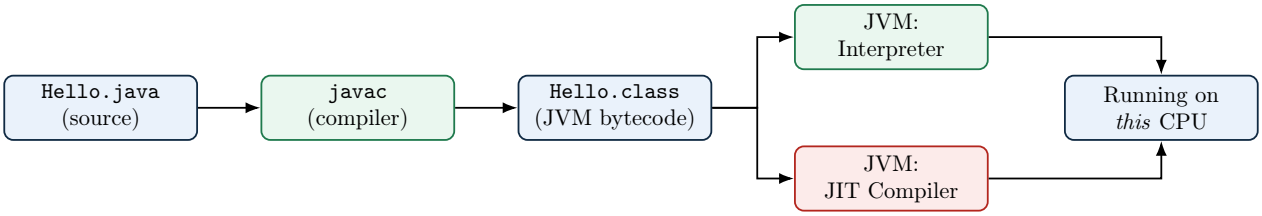
[beyond DB] Java is the single clearest real-world illustration of everything in §1.2, so it is worth walking through as a complete case study — especially since Java's own design goals were *explicitly* about the compiler/interpreter trade-off from the start.

1.2.5.1 From Pure Interpreter to JIT: A Short History

Era	What Happened
Java 1.0 (1996)	The original JVM was a **pure bytecode interpreter**. Every bytecode instruction was fetched, decoded, and dispatched on every single execution – simple and portable, but roughly 10–20× slower than equivalent compiled C, which badly hurt Java's early reputation for performance.
Java 1.2 – "HotSpot" research	Sun's research team built an adaptive compiler codenamed **HotSpot**: instead of compiling everything, or nothing, it would *watch the program run* and compile only the small fraction of methods that actually matter for performance.
Java 1.3 (2000) onward	HotSpot became the default production JVM. This is the point Java became a genuine **hybrid**: start by interpreting (fast startup), then JIT-compile whichever methods turn out to be "hot" (fast steady state). Every mainstream JVM since has kept this same architecture.

1.2.5.2 Compilation to Bytecode, Then Portability via the JVM

[DB] `javac`, the Java *compiler*, does not produce machine code for any real CPU at all. It produces **Java bytecode** – instructions for an abstract, idealized machine called the **Java Virtual Machine (JVM)** that does not physically exist as hardware.



"Write Once, Run Anywhere"

Because `javac` only ever has to target *one* abstract machine (the JVM specification) instead of every physical CPU family, the exact same `Hello.class` file runs unmodified on Windows/x86, Linux/ARM, or a JVM on any other platform — as long as *that* platform has a compliant JVM installed. All the platform-specific work is pushed into the JVM implementation itself (one per platform, built once by the JVM vendor), instead of into every single Java program ever written. This is Sun Microsystems' famous "Write Once, Run Anywhere" pitch for Java, and it is the exact same $M + N$ argument (rather than $M \times N$) you will meet again with LLVM IR in Lecture 2 — just applied to distribution instead of to compiler internals.

1.2.5.3 HotSpot's Tiered JIT in More Depth

[DB] Modern HotSpot uses **tiered compilation**, layering the interpreter and two JIT compilers so the JVM is never stuck doing more work than the code's importance justifies:

1. Every method starts out **interpreted**, while the JVM silently maintains an **invocation counter** (calls) and a **back-edge counter** (loop iterations) for it.
2. Once a counter crosses a threshold, the **C1 (client) compiler** quickly produces a lightly-optimized native version, and also inserts **profiling instrumentation** into it (recording which branches are taken, which concrete types show up at a call site, ...).
3. If the method keeps getting hotter, the **C2 (server) compiler** recompiles it a second time, now using the profiling data gathered by the C1 version to make aggressive, *speculative* optimizations: inlining a virtual method call that has – so far – always resolved to the same concrete type; eliminating a heap allocation entirely if it can prove the object never escapes the method (**escape analysis**); unrolling a hot loop.
4. If a speculative assumption is later violated (a second, different type shows up at that call site), HotSpot **deoptimizes**: it discards the C2 code for that method and falls back to the interpreter (or C1) until it is confident enough to try recompiling again.

Exam Focus

Be able to explain, in your own words, why HotSpot's C2 compiler can safely make optimizations (like assuming a call site's type never changes) that a purely static AOT compiler like GCC never could – and what mechanism (deoptimization) lets it recover if that assumption turns out to be wrong.

1.2.5.4 JIT vs. AOT vs. Interpreter: Advantages and Disadvantages

Pulling together everything from the enhanced table in §1.2.4 and the HotSpot walkthrough above, here is the direct trade-off summary for a JIT compiler specifically:

JIT Advantages	JIT Disadvantages
Can use **real runtime profile data** (actual types, actual branch frequencies) to make optimizations no static compiler can safely attempt – generally impossible for an AOT compiler, which only ever sees the source.	**Warm-up latency**: peak performance is only reached after enough executions to get hot code recompiled – a real problem for short-lived programs (e.g. CLI tools, serverless functions).
Ships one **portable** bytecode artifact instead of a separate binary per target CPU – matching an interpreter's portability, unlike AOT.	Consumes **extra CPU and memory at run time** for profiling counters, the JIT compiler threads themselves, and a generated-code cache – resources an AOT binary or a plain interpreter never spend.
Reaches **steady-state speed close to AOT-compiled native code** for long-running, hot-path-dominated programs – far faster than a pure interpreter ever gets.	**Less predictable** performance moment-to-moment (a method might pause execution while it gets recompiled), which matters for hard real-time systems that AOT compilation suits better.
Can **adapt to the actual host CPU** at run time (e.g. use available vector instructions) without shipping separate binaries per microarchitecture, unlike a single AOT build.	**Significantly more implementation complexity**: an interpreter, multiple JIT tiers, profiling instrumentation, and deoptimization machinery, versus AOT's comparatively simple "compile once, done."

Extra Depth: The Java Class File, Briefly

A `.class` file is itself a small, well-defined binary format worth knowing at a glance: it opens with a **magic number** (`0xCAFEBABE`, a Sun Microsystems in-joke), a class-file version number, a **constant pool** (a table holding every literal, class name, and method/field reference the class uses – similar in spirit to the symbol table we will meet in Lecture 2), and then the actual bytecode for each method. Tools like `javap -c Hello.class` let you disassemble a compiled class and read its raw JVM bytecode instructions directly – a nice hands-on way to see §1.2.3's "token stream → bytecode" pipeline made completely concrete.

1.3 Why Study Compilers? (DB §1.1)

[DB] You may never write a production C compiler. So why is this course core, and why does almost every strong CS program require it? Three overlapping reasons:

1. **Compilers are, in DB's phrase, "one of the largest and most complex software systems" commonly built** – yet a working one is buildable in a single semester project. Nowhere else in the curriculum will you build something this large, this precisely specified, end to end, from a formal problem statement to a working artifact.
2. **The techniques generalize far beyond compilers.** Finite automata power text editors, network protocol parsers, and input validation. Context-free grammars appear in JSON parsers, config-file readers, and query languages. Data-flow analysis underlies static analyzers, IDE "find usages" features, and security vulnerability scanners. You are not just learning to build a compiler; you are learning the standard toolkit for building *any* system that must process structured, language-like input reliably.
3. **Compilers sit at the hardware/software boundary.** Understanding how a `for` loop becomes register moves and branch instructions demystifies performance – why one piece of "equivalent" code runs 10× faster than another, why cache-friendly code matters, and why some

languages can be faster than others for reasons that have nothing to do with the programmer's skill.

Extra Depth: A "Meta" Argument

There's a fourth reason DB does not spell out explicitly but that experienced engineers often cite: compiler construction is one of the few courses where you are forced to build a *multi-stage pipeline* with cleanly separated concerns (lexing, parsing, semantic analysis, code generation) and *formally specified* correctness at each stage. This is directly transferable to designing any large pipeline system – data pipelines, network stacks, even ML training pipelines – where the same discipline of "define an intermediate representation, verify it, transform it, repeat" shows up again and again.

1.4 Applications of Compiler Technology (DB §1.5)

[DB] DB is explicit that compiler techniques reach "far beyond" translating a general-purpose programming language into machine code. Five application areas DB highlights:

1. **Implementation of high-level programming languages.** The obvious case, but note the historical point DB makes: higher-level languages raised the level of abstraction precisely *because* compiler technology matured enough to make that abstraction affordable in performance terms.
2. **Optimizations for computer architectures.** As hardware has grown more parallel (multicore, SIMD/vector units, deep pipelines, GPUs), the compiler has taken on more responsibility for finding parallelism and hiding memory latency that programmers cannot realistically manage by hand.
3. **Design of new computer architectures.** Hardware designers now co-design instruction sets *with* the compiler in mind (RISC philosophy: keep hardware simple, let the compiler do the scheduling), since a feature only helps if a compiler can reliably exploit it.
4. **Program translations.** Beyond source-to-machine-code, DB lists: binary translation (retargeting compiled binaries to a new instruction set, e.g. Apple's Rosetta 2 translating x86 binaries to run on Apple Silicon), hardware synthesis (compiling a hardware description language like Verilog into actual circuit layouts), database query interpreters (translating SQL into an execution plan), and compiled simulation (compiling a simulation model, e.g. for chip verification, into fast executable code rather than interpreting it).
5. **Software productivity tools.** Type checking, bounds checking, and static program-analysis tools that catch bugs before run time all reuse the same front-end machinery (lexers, parsers, semantic analyzers) that compilers use.

Extra Depth: Applications DB Does Not Mention

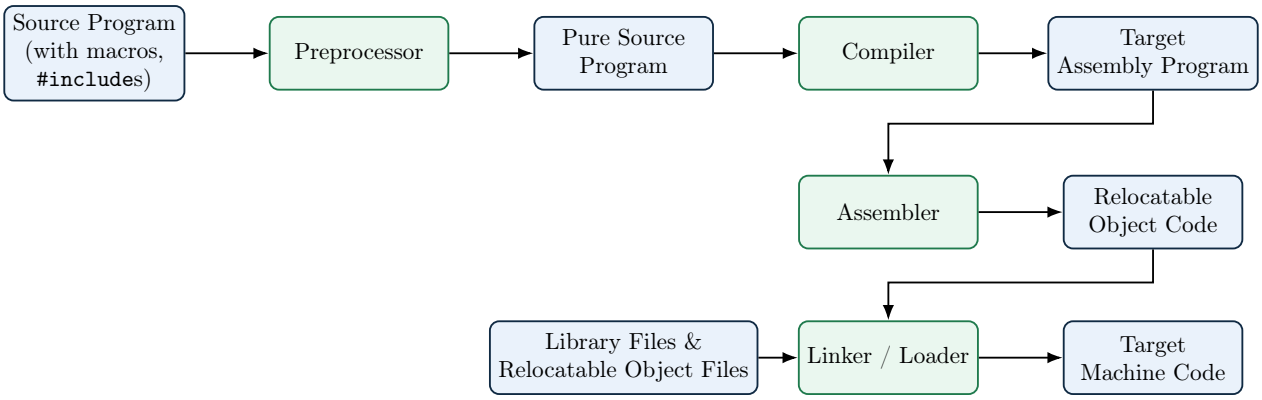
The five areas above were written with early-2000s computing in mind. Several major compiler-technology application areas have grown enormously since and deserve a mention:

- **Machine-learning compilers.** Frameworks like Google's XLA, Apache TVM, and MLIR (Multi-Level Intermediate Representation, itself built using classic compiler theory) compile neural-network computation graphs down to highly optimized GPU/TPU kernels – this is now one of the most active areas of compiler research.
- **WebAssembly (WASM).** A portable, sandboxed bytecode target designed so *any* source language (C, Rust, Go, ...) can compile to it and run safely inside a browser or standalone runtime at near-native speed.

- **Superoptimizers.** Tools (e.g. Souper, STOKE) that search – sometimes with SMT solvers, sometimes with reinforcement learning – for provably optimal short instruction sequences, going beyond what traditional rule-based optimization passes can find.
- **Domain-specific languages (DSLs) and their compilers.** Halide (image-processing pipelines), SQL query planners, regular-expression engines, and shader languages (GLSL/HLSL for GPUs) are all compilers for a language deliberately kept small and specialized so it can be optimized far more aggressively than a general-purpose language could be.
- **Security applications.** Control-flow integrity instrumentation, automatic bounds-check insertion (AddressSanitizer), and fuzzing harness generation are compiler-pass techniques applied specifically to hardening software against attack.
- **LLVM as shared infrastructure.** Perhaps the biggest structural change since DB 2nd edition: LLVM lets dozens of front-ends (Clang/C/C++, Rust, Swift, Julia, and more) share one very well-engineered optimizer and set of back-ends, instead of every language reinventing code generation. We will return to LLVM in Lecture 2.

1.5 A Typical Language-Processing System

[DB] The "compiler" most people invoke by typing `gcc file.c` is not, strictly, one single program at all – it is a **driver** that coordinates several distinct tools in sequence. DB places the compiler proper inside this larger **language-processing system**.



A typical language-processing system (compare DB Fig. 1.5): the "compiler" you invoke on the command line is really a pipeline of five distinct programs.

- **Preprocessor.** Expands macros (`#define`), includes files (`#include`), and resolves conditional-compilation directives (`#ifdef`) — all as pure text substitution, before the compiler's grammar-aware phases ever run.
- **Compiler (proper).** Everything from §1.2: source → target assembly (today's "black box"; Lecture 2 opens it up phase by phase).
- **Assembler.** Translates human-readable assembly instructions into **relocatable machine code** and packages it as an **object file**.
- **Linker.** Combines multiple object files and library files into one program, resolving references between them.
- **Loader.** Places the final program into memory and starts it running.

Is an Assembler a Compiler?

By the definition in §1.1, yes: an assembler reads a program in one language (assembly) and translates it into an equivalent program in another language (relocatable machine code), with no execution of its own along the way – that is exactly the compiler definition. It is given its own name and its own box in the pipeline diagram above for practical, not definitional, reasons: assembly is designed to be an almost line-for-line, one-to-one notation for machine instructions, so an assembler's job skips most of what makes a "full" compiler hard (there is essentially no analysis phase, no optimization search, and little structural gap between source and target to bridge) and reduces mostly to table lookup and address bookkeeping. The lesson generalizes: *compiler* names a *role* – translate one language into another, without executing either – not a claim about how sophisticated the translation has to be. An assembler, a JVM-bytecode compiler like `javac`, and a heavily-optimizing tool like GCC are all compilers in exactly the same technical sense; they differ only in how much analysis and transformation the translation requires.

Definition: Object File

An **object file** is the output of the assembler: a binary file containing machine instructions and data *that is not yet a runnable program.* Critically, it contains:

- **Relocatable code/data**, with memory addresses left as placeholders, since the assembler does not know where in memory this code will ultimately live;
- A table of **symbols this file defines** (e.g. a function it implements, available for other files to call); and
- A table of **symbols this file references but does not define** (e.g. a call to `printf`, defined elsewhere in the C standard library) – these are exactly what the linker must resolve next.

On Linux this is the `.o` file produced by `gcc -c`; the same idea is called an `.obj` file on Windows.

What the Linker Actually Does

Programs are almost never compiled as a single file. The **linker**'s job is to take *several* object files (and pre-built library files) and merge them into one program, by:

1. **Symbol resolution:** matching every symbol a file *references* (like a call to a function defined in another file, or in a library) to the file that actually *defines* it. An unresolved symbol here is exactly the "undefined reference to `foo`" error you see at link time, as distinct from a compiler error.
2. **Relocation:** now that every piece of code has been assigned a real, final address in the combined program, the linker patches every placeholder address left by the assembler with the correct final value.

The output is one combined file — an **executable** — with (in the simplest case) no unresolved symbols and no remaining placeholder addresses.

What the Loader Actually Does

The **loader** is typically part of the operating system, not the compiler toolchain. When you run a program, the loader:

1. Allocates memory for the program (code, data, stack, heap regions);

2. Copies (or memory-maps) the executable's instructions and initial data into that memory;
3. Performs any *final* address fix-ups that were deferred until this exact moment (relevant for dynamically linked libraries, previewed in Lecture 2); and
4. Transfers control to the program's entry point – the actual first instruction that runs, which is *not* always your `main` function directly (a small language-runtime startup routine usually runs first).

Are Addresses Absolute After Linking, or Only After Loading?

It depends on which of two linking models is used, and modern systems mostly use the second one.

- **Fixed-base static linking (the "simplest case" above).** The linker assumes the program will always be loaded at one predetermined virtual address. Given that assumption, the linker already has everything it needs to compute true, final **absolute** addresses during linking itself – the "Relocation" step above is the linker permanently writing those absolute values into the executable. The loader's job then is just to copy bytes into memory at that fixed spot; no address math happens at load time.
- **Position-independent executables (PIE) and shared libraries – the modern default.** A shared library (`.so`/`.dll`) must be loadable at different addresses in different processes, since many unrelated programs share one copy of it; and ASLR (address space layout randomization) deliberately loads a program at a different random base address on every run, as a security defense. In both cases the linker *cannot* know the final load address in advance, so it deliberately leaves references as **relative** offsets (or entries in a relocation table) instead of fixing them. Only at load time, once the loader's dynamic-linking component has actually chosen a base address for that run, do those relative offsets become true run-time absolute addresses – this is exactly the "final address fix-ups" mentioned in step 3 above.

So "linker addresses are relative and become absolute once the loader picks where the program is loaded" is correct specifically for the PIE/shared-library case, which is now the common one – not a universal property of everything the linker produces.

Exam Focus

A very common exam trap: "which phase reports an 'undefined reference' / 'unresolved external symbol' error for a missing function?" The answer is the **linker**, not the compiler — the compiler is perfectly happy as long as a function is *declared* (e.g. via a header file); only the linker checks that a matching *definition* actually exists somewhere among all the object files and libraries it was given.

Extra Depth: Seeing the Real Pipeline on Your Own Machine

You can watch this exact pipeline happen with GCC or Clang by stopping it at each stage:

- `gcc -E prog.c -o prog.i` — run *only* the preprocessor; inspect `prog.i` to see macros expanded and headers inlined.
- `gcc -S prog.i -o prog.s` — run the compiler proper; `prog.s` is human-readable target assembly.

- `gcc -c prog.s -o prog.o` — run the assembler; `prog.o` is a relocatable ELF (on Linux) object file. Inspect it with `objdump -d prog.o` or `nm prog.o`.
- `gcc prog.o -o prog` — run the linker, pulling in the C runtime and any libraries, producing the final executable.

Try this on a one-line `"hello world"` program: the jump in size and content between `prog.i` (hundreds of lines, from expanded standard-library headers) and `prog.c` (one line) is a genuinely eye-opening way to internalize what the preprocessor actually does.

Extra Depth: Static vs. Dynamic Linking, and What the Loader Really Does

The linker/loader description above is simplified; two refinements matter a great deal in practice:

- **Static linking** copies the code of every library function you use directly into your executable at link time. The result is self-contained but larger, and does not benefit when the OS wants to share one copy of, say, `libc`, across many running programs.
- **Dynamic linking** instead records, in the executable, only the *names* of needed shared libraries (`.so` on Linux, `.dll` on Windows, `.dylib` on macOS). A specialized program, the **dynamic linker/loader** (`ld.so` on Linux), runs as the executable starts, finds each shared library already resident in memory or maps it fresh, and **patches in** the real addresses of library functions — a process called **relocation** — before your `main` function ever runs. This is why dynamically linked executables are smaller on disk but have slightly higher startup latency and a runtime dependency on the right library versions being present (colloquially, "DLL hell").
- **Position-Independent Code (PIC)** is machine code generated so it can be loaded at *any* memory address, not a fixed one — essential for shared libraries (which may be mapped at different addresses in different processes) and for security features like ASLR (Address Space Layout Randomization). Generating PIC is a back-end code-generation decision, another example of how deep the "compiler" choices reach into topics normally considered OS territory.

Extra Depth: Build Systems Are Compiler Orchestration at Scale

Everything above describes compiling *one* file. Real projects have thousands. The tools that decide *which* files need recompiling and in what order are themselves built on the phase model above:

- **Make** / **Ninja** track file modification timestamps and a dependency graph to avoid recompiling unchanged files — essentially caching the output of the "compiler" node in the pipeline diagram above, keyed by source-file identity and timestamp.
- **ccache** / **sccache** go further: they hash the *preprocessed* source (post-preprocessor, pre-compiler) plus compiler flags, and cache the resulting object file, so an unchanged file compiled with unchanged flags never re-invokes the compiler proper at all — directly exploiting the preprocessor/compiler boundary described earlier in this section.
- **Bazel and other hermetic build systems** extend caching across an entire team or CI fleet: if any engineer, anywhere, has already built a given (source, flags, toolchain) combination, everyone else can fetch the cached object/executable instead of recompiling.
- **Link-Time Optimization (LTO / ThinLTO)** deliberately *breaks* the strict separate-compilation model: instead of each `.c` file being optimized in isolation before linking, LTO defers whole-program optimization to link time, when the optimizer can see every

translation unit at once and, e.g., inline a function across what used to be file boundaries. This directly trades the modularity of the pipeline in §1.5 for better runtime performance.

Real Object-File Formats – Beyond This Introduction

This lecture's treatment of object files, linking, and loading is deliberately introductory. Real object-file formats – ELF on Linux, Mach-O on macOS, PE on Windows – each encode the relocatable code/data, symbol tables, and section layout described above in a fully specified binary format, with their own tools (`readelf`, `otool`, `dumpbin`) for inspecting them. That level of detail is not covered in this course, but is worth exploring on your own with the `objdump`/`readelf` commands mentioned above.

1.6 Different Kinds of Compilers

[beyond DB] "Compiler" is used as an umbrella term across the industry for several genuinely different tools, distinguished by *what kind of language* sits on each side of the translation (high-level vs. low-level) and *which machine* runs the result. Six worth knowing by name:

Kind	Source → Target	Example(s)
Native (AOT) compiler	High-level → machine code, same target as the host machine	GCC, Clang, `rustc` compiling for the machine they run on
Source-to-source compiler (transpiler)	High-level → a *different* high-level language	TypeScript → JavaScript; Babel (modern JS → older JS); CoffeeScript → JavaScript
Cross-compiler	High-level → machine code for a target *different* from the host running the compiler	Compiling ARM binaries for a phone on an x86 laptop; embedded firmware toolchains
Binary translator	Low-level (one instruction set) → low-level (a *different* instruction set)	Apple's Rosetta 2 (x86-64 → ARM64); QEMU's user-mode emulation
Decompiler	Low-level (machine code or bytecode) → high-level-*like* source	Ghidra, IDA Pro (machine code); JD-GUI, CFR (Java bytecode)
Bootstrapping (self-hosting) compiler	A compiler for language X, *written in* X itself	GCC (mostly C); `rustc` (written in Rust)

Example: Two of These, Worked Through

Binary translation (Rosetta 2): when Apple moved Macs from Intel (x86-64) to Apple Silicon (ARM64) in 2020, millions of existing apps were compiled x86-64 binaries with no source available to the end user. Rosetta 2 translates those already-compiled x86-64 binaries into ARM64 machine code – notice both sides of this translation are *low-level*, which is what makes

it a binary translator rather than an ordinary compiler.
Bootstrapping (GCC): you cannot compile the very first C compiler *using* a C compiler, since none exists yet. Historically, an initial, minimal C compiler was written in assembly or an even earlier language; once that minimal compiler could compile *a subset* of C, a fuller C compiler was written in C and compiled by the minimal one; that fuller compiler then compiles *itself* (and every subsequent version) from then on. This chicken-and-egg-breaking process is exactly what "self-hosting" means.

Exam Focus

Given a described tool ("translates Kotlin to Java source," "translates ARM machine code back into readable pseudo-C," "compiles Swift for an iPhone from a Mac"), be able to name which of these six kinds it is and justify your answer using the *source* → *target* column above – this classification-by-example is a very common short-answer format.

Where the Compiler-vs-Interpreter and Kind-of-Compiler Axes Meet

These two classification schemes are independent and can combine freely. A JIT compiler *is* a native compiler by this section's scheme (high-level/bytecode → machine code), just one whose *timing* (during execution) differs from an AOT native compiler's. A transpiler like Babel is almost always itself an AOT tool – it finishes its work entirely before the JavaScript it produces ever runs. Don't confuse "what kind of translation is this" (this section) with "when does the translation happen" (§1.2) – exam questions sometimes test both axes on the same tool deliberately.

1.7 A Preview: What's Next

Lecture 2 opens up the compiler "black box" from §1.2 of this lecture and walks through its internal phases – lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and final code generation – in real depth, including how the phases group into a front-end, middle-end, and back-end. Everything in today's lecture is about *what* these systems do from the outside; Lecture 2 begins answering *how*, in detail.

1.8 Self-Check Questions

Practice – Not Graded

1. In one sentence, explain why "a compiler executes your program" is a factually incorrect statement, and what does execute it instead.
2. Classic CPython has no JIT; PyPy does. Predict, with a reason, which one would be faster for a program consisting of a single loop that runs 50 million times, versus a script that runs once and exits immediately.
3. Explain, using the $M + N$ vs. $M \times N$ argument, why compiling to JVM bytecode makes Java portable in a way that compiling directly to x86 machine code would not.
4. A program compiles successfully with no errors, but fails at link time with "undefined reference to `compute_total`." Which tool caught this, and why couldn't the compiler have caught it earlier?
5. Is a transpiler (e.g. TypeScript → JavaScript) a compiler under DB's definition? Justify

your answer using the definition given in §1.1, and separately, name which of the six "kinds of compiler" from §1.6 it belongs to.

6. Give one example, not listed in this lecture, of a real tool on your own laptop that is best described as an "interpreter" rather than a compiler.
7. HotSpot's C2 JIT only compiles a method after it has been *interpreted* many times. Why not just JIT-compile every method the very first time it's called?

Key Takeaways

- A **compiler translates**; it does not itself execute the source program. An **interpreter** executes the source directly, with no independent target program left behind.
- Compilers trade slower build time for faster, repeatable run time, at the cost of a target binary that is **not portable** (tied to one ISA/OS) and paid for by the whole-program visibility that enables aggressive, multi-pass optimization. Interpreters trade that away for instant startup, easy interactivity, and **source-level portability**, at the cost of repeated re-interpretation overhead on every run.
- Most real systems today (JVM, V8, .NET, PyPy) are **hybrids**: compile to portable bytecode, interpret it at first, then **JIT-compile** the hot paths to native code at run time – Java's own history (pure interpreter in 1.0, HotSpot's tiered JIT from 1.3 onward) is the clearest real example, and it's what makes "write once, run anywhere" work.
- A JIT's big edge over AOT is **using real runtime profile data** for speculative optimization; its costs are **warm-up latency** and **runtime profiling/compilation overhead** – know this trade-off table cold.
- The compiler proper sits inside a larger **language-processing system**: preprocessor → compiler → assembler → linker → loader. The assembler produces an **object file** (relocatable code plus defined/referenced symbol tables); the **linker** resolves symbols and relocates addresses; the **loader** places the program in memory and starts it.
- "Compiler" covers several genuinely different tools: **native/AOT compilers**, **source-to-source transpilers**, **cross-compilers**, **binary translators**, **decompilers**, and **bootstrapping/self-hosting compilers** – know the source→target shape of each.
- Compiler techniques (automata, grammars, data-flow analysis) show up far outside traditional compilers – in IDEs, static analyzers, database engines, and ML frameworks – which is the real reason this course is core, not elective.
- DB §1.5's five application areas (language implementation, architecture optimization, architecture design, program translation, productivity tools) have since been joined by ML compilation, WebAssembly, and shared infrastructure like LLVM.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 2

Structure of a Compiler: Phases Overview; Front-/Middle-/Back-End; MLIR and Real-World IRs

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.2 (The Structure of a Compiler), §2.1–§2.8 (overview of a syntax-directed translator's stages)
Other references:	[PP] Programming Language Pragmatics §1.5, §1.6

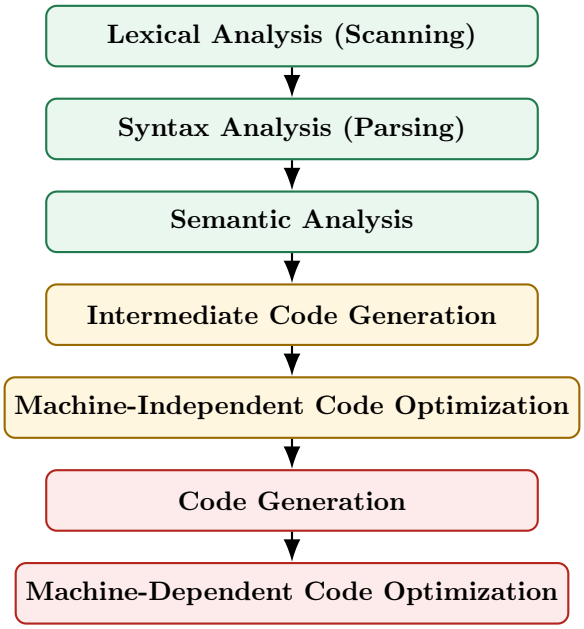
Lecture Roadmap

1. The classical phase pipeline of a compiler (DB Fig. 1.6): what each phase consumes and produces, followed all the way through on one tiny worked example.
2. The two cross-cutting components every phase touches: the **symbol table** and **error handling**.
3. Grouping the phases into **front-end** / **middle-end** / **back-end**, and why that grouping is an engineering decision, not just a diagram convenience.
4. How real-world intermediate representations – **LLVM IR, JVM bytecode, WebAssembly, and MLIR** – relate to this pipeline and to each other.

Today is still "the outside view." From Lecture 3 onward we go phase-by-phase in depth, starting with lexical analysis. (The preprocessor/assembler/linker/loader toolchain around the compiler was already covered in Lecture 1.)

2.1 The Phase Pipeline of a Compiler

[DB] DB organizes a compiler as a pipeline of phases, each of which takes the output of the previous phase as input, checks it, and produces a new representation of the program one step closer to executable code.



Phase	Input	Output
Lexical Analysis	Raw character stream	Stream of **tokens** (e.g. `ID`, `NUM`, `+`)
Syntax Analysis	Token stream	**Syntax tree** (or parse tree) capturing grammatical structure
Semantic Analysis	Syntax tree	Annotated ("decorated") tree: types checked, scopes resolved
Intermediate Code Gen.	Annotated tree	Machine-independent **IR**, e.g. three-address code
Code Optimization	IR	Improved, semantically-equivalent IR
Code Generation	Optimized IR	Target-machine code (often assembly)

Exam Focus

Memorize the phase order and, for each phase, one sentence describing its input and output. This exact table (input → phase → output) is the single most common short-answer / MCQ pattern in compiler exams worldwide, not just here.

2.1.1 A Tiny Example, Followed Through Every Phase

[DB] It helps enormously to see one small piece of source code survive the *entire* pipeline, phase by phase, before we study each phase in isolation over the coming lectures. DB uses exactly this running example when it first introduces the phases (§1.2); we reproduce a miniature version of it here, and will point back to it by name ("the `position` example") whenever a later lecture revisits one of these phases in more depth. Nothing here needs to be memorized yet – it is a preview, not new material to be tested on its own.

Source statement (assume `position`, `initial`, and `rate` are declared as real/floating-point variables, and `60` is an ordinary integer literal):

```
position = initial + rate * 60
```

Definition: Token and Lexeme

A **lexeme** is the actual sequence of characters in the source program that the lexical analyzer recognizes as a single meaningful unit – for example, the seven characters `p-o-s-i-t-i-o-n` form one lexeme, and the single character `=` forms another. A **token** is the abstract `<token-name, attribute-value>` pair the lexer emits once it has recognized a lexeme: the token-name records *which category* was matched (e.g. `ID`, `ASSIGN`, `PLUS`), and the attribute-value carries whatever extra information later phases need in order to tell this particular lexeme apart from other lexemes in the same category – for an identifier, almost always a pointer into the symbol table. Lecture 3 gives the full formal treatment, including **patterns** – the rules that describe which lexemes belong to which token.

For this statement, the **lexemes** are the individual pieces of source text `position`, `=`, `initial`, `+`, `rate`, `*`, and `60`. Lexical analysis classifies each one as a **token**: the lexeme `position` becomes `<ID, ptr to symtab entry for position>` – written `<id,1>` in the walkthrough below – and likewise `initial` and `rate` become `<id,2>` and `<id,3>`; the lexeme `=` becomes an `<ASSIGN>` token, `+` a `<PLUS>` token, `*` a `<MULT>` token, and `60` a `<NUM, 60>` token. None of the three operator tokens need an attribute-value, since the token-name alone already tells the parser everything it needs to know.

Example: The `position` Statement, Phase by Phase

- **Lexical analysis (Lectures 3–7).** The character stream is grouped into tokens: `<id,1>` `<=>` `<id,2>` `<+>` `<id,3>` `<*>` `<60>` – where `id,1`, `id,2`, and `id,3` are pointers to the

symbol-table entries for `position`, `initial`, and `rate` respectively.

- **Syntax analysis (Lectures 8–17).** The token stream is organized into a syntax tree that reflects the grammatical structure of an assignment: an `=` node whose left child is `id,1` and whose right child is a `+` node, which in turn has children `id,2` and a `*` node with children `id,3` and `60`.
- **Semantic analysis (Lectures 18–21).** Types are checked, and where necessary, made explicit. Here, `60` is an integer literal but `rate` is real, so the semantic analyzer inserts an explicit conversion: the `*` node's right child becomes `inttoreal(60)` instead of the bare integer `60`. The result is an *annotated* ("decorated") tree.
- **Intermediate code generation (Lectures 22–25).** The annotated tree is translated into three-address code, one operation per instruction:

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

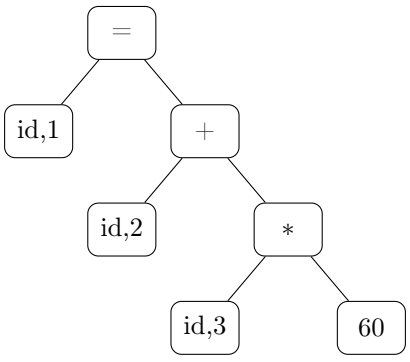
- **Code optimization (Lectures 28–29).** `inttoreal(60)` does not depend on any variable, so it can be computed once, at compile time, instead of being recomputed by the generated program every single time this line runs:

```
t1 = id3 * 60.0
id1 = id2 + t1
```

- **Code generation (Lecture 30).** The optimized IR becomes target assembly, using real machine registers and instructions (illustrative syntax, not tied to one specific ISA):

```
MOVF   id3, R2
MULF   #60.0, R2
MOVF   id2, R1
ADDF   R2, R1
MOVF   R1, id1
```

The syntax-analysis step above builds the following syntax tree – the root `=` node assigns to `id,1` the value computed by its right subtree:



Syntax tree for `position = initial + rate * 60`.

Exam Focus

This example is deliberately small enough to hold in your head. When a later lecture introduces the *real* algorithm for a phase (Thompson's construction, LALR(1) table construction, available-expressions data-flow analysis, and so on), try re-running that algorithm on this same statement

by hand before tackling a harder one – it is a fast way to sanity-check that you have understood a new algorithm correctly.

2.1.2 Analysis and Synthesis: The Two Halves

[DB] DB groups these six phases into two conceptual halves:

- **Analysis part ("front end"):** lexical, syntax, and semantic analysis, plus intermediate code generation. This half *breaks down* the source program and builds an intermediate representation plus a record of the program's structure (the symbol table). Analysis is where most **compile-time errors** are caught and reported.
- **Synthesis part ("back end"):** optimization and code generation. This half *constructs* the desired target program from the intermediate representation and symbol table.

Definition: Analysis-Synthesis Model

A compiler can be understood as consisting of an **analysis** part, which breaks the source program into pieces and builds an intermediate representation, and a **synthesis** part, which constructs the target program from that intermediate representation. This is DB's own high-level characterization of *every* compiler, regardless of source or target language.

2.2 Symbol Table and Error Handling: The Cross-Cutting Phases

[DB] Two components are not steps in the pipeline — they run *alongside every phase*, being read from and written to throughout the whole compilation: the **symbol table** and **error handling**.

Definition: Symbol Table

A **symbol table** is a data structure that records every identifier declared in a program – variables, function names, class names – together with the attributes later phases need to know about it: its **type**, **scope**, **memory location**, and, for functions, its parameter and return types. It is not itself one of the six phases; instead every phase reads from it and writes to it: lexical analysis creates an entry the first time it sees a new identifier, semantic analysis fills in type information once it is known, and code generation consults it to compute final memory addresses.

Example: Symbol Table Contents for the `position` Example

For `position = initial + rate * 60` (§2.1.1), lexical analysis creates one entry per distinct identifier – exactly the `id,1`, `id,2`, and `id,3` pointers used throughout that walkthrough:

Entry	Name	Type	Role in this statement
`id,1`	`position`	real	assigned to
`id,2`	`initial`	real	read
`id,3`	`rate`	real	read

Notice `60` never gets its own entry – it is a numeric *literal*, not an identifier, so it is carried as a token attribute (`<NUM, 60>`) rather than being recorded in the symbol table.

Error handling. Each phase may encounter errors specific to its own level: lexical analysis catches malformed tokens, syntax analysis catches grammatically invalid programs, semantic analysis catches type mismatches and undeclared identifiers. A well-built compiler tries to *recover* from an error and keep going, so it can report several distinct errors from a single compilation run instead

of stopping at the first one.

2.2.1 Lexical, Syntax, and Semantic Errors

[DB] The three kinds of errors above are easy to name but easy to blur together in practice. Getting the distinction right matters because it tells you *which phase* is responsible for catching a given mistake – and, just as importantly, which phases are *not* responsible for it and will happily let it through.

Definition: Lexical, Syntax, and Semantic Errors

- A **lexical error** occurs when the lexer meets a character sequence that matches *no pattern for any token at all* – e.g. an illegal character, an unterminated string or comment, or a malformed numeric literal. The lexer fails before the parser ever sees this part of the input.
- A **syntax error** occurs when the token stream – with every individual token itself perfectly valid – does not conform to the grammar: e.g. a missing semicolon, an unbalanced parenthesis, or an operator with no right-hand operand.
- A **semantic error** occurs when the program is grammatically well formed but violates a rule about *meaning* that the language enforces: e.g. using an undeclared identifier, assigning incompatible types, or calling a function with the wrong number of arguments.

Each is caught by a different phase, in this order, because each phase can only see what the phase before it has already built: lexical analysis sees individual characters, syntax analysis sees the shape of the token stream, and semantic analysis is the first phase that understands what the program actually *means* (using the symbol table above to check declarations, types, and scope).

Example: One Error of Each Kind, in One Function

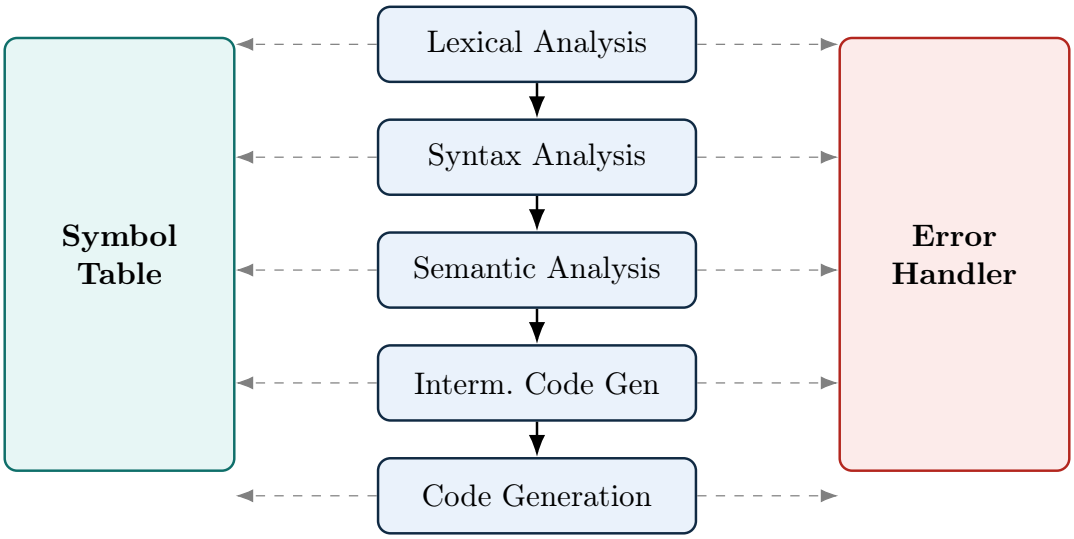
```
int main() {
    int count = 0;
    int limit = 10           /* (1) */

    while (count < limit) {
        count = count # 1;   /* (2) */
        total = total + count;   /* (3) */
    }

    printf("Final count = %d", count);
    return 0;
}
```

Line	Error Kind	Why
(1)	Syntax	`int limit = 10` is missing the terminating `;`. Every token on this line is individually a valid token – the parser is the one that fails, because it cannot complete the grammar rule for a declaration statement without a semicolon before the next statement begins.
(2)	Lexical	`#` is not a valid character inside a C expression. No token pattern matches it at all, so the *lexer itself* fails right here – the parser never even gets a chance to see this line.
(3)	Semantic	`total` is used but was never declared anywhere in this function. Every token is valid and the statement is a grammatically perfect assignment, so only semantic analysis – which checks the symbol table for declarations and scope – can catch this.

Notice the ordering this implies: if line (2)'s illegal character were fixed but line (1)'s missing semicolon were not, the compiler would still stop at the syntax error before ever reaching semantic analysis and reporting the undeclared `total` on line (3) – each phase only gets to run once the phase before it has accepted its input.

Exam Focus

Given a short, broken code snippet, be able to classify each mistake as lexical, syntax, or semantic, and justify the classification the way the table above does – this is one of the most common short-answer formats for this topic. A frequent trap: a misspelled *keyword* (like `fi` for `if`) is **not** a lexical error, because it is still a perfectly valid lexeme for token `ID` – the mistake only surfaces later, as a syntax error (if `fi` appears somewhere no identifier is grammatically valid) or a semantic error (if it is used as if it were a declared function or variable).



Extra Depth: What Real Symbol Tables Store

A production compiler's symbol table stores far more than DB's introductory description suggests: mangled names (for overload resolution in C++/Rust), calling-convention information, alignment and padding requirements, visibility/linkage (`static` vs. exported), inlining hints, and — in languages with generics/templates — a separate table per instantiation. Modern compilers

often implement it not as one flat table but as a **chain of nested scopes** (a scope stack or scope tree), so that entering a block, function, or class pushes a new scope that shadows outer ones and is popped on exit. LLVM's `clang` keeps this information in its `Decl` hierarchy rather than a single table, reflecting how much richer real symbol-table designs are compared to the simple table DB introduces.

2.3 Grouping the Phases: Front-End, Middle-End, Back-End

[beyond DB] DB's own diagram groups phases into analysis/synthesis, but the vocabulary you will hear constantly in industry and in later lectures is **front-end** / **middle-end** / **back-end**. This is not just relabeling — it reflects a real engineering strategy.

Group	Contains / Responsibility
Front-end	Lexical, syntax, semantic analysis. Language-*specific*: knows the grammar and type rules of exactly one source language, and lowers it to a common IR.
Middle-end	Machine-independent optimization on the IR. Language-*agnostic* and target-*agnostic*: the same optimization passes (constant folding, dead-code elimination, inlining) apply no matter which source language or target CPU is involved.
Back-end	Code generation and machine-dependent optimization. Target-*specific*: knows the instruction set, register file, and calling convention of exactly one target architecture.

Why This Grouping Is the Whole Point

If front-ends only ever talk to a shared IR, and back-ends only ever talk to that same shared IR, then supporting a new *source language* only requires writing one new front-end — it automatically gets every optimization and every target the IR already supports. Symmetrically, supporting a new *target CPU* only requires writing one new back-end — it automatically works for every source language already supported. Without this split, you would need to write a separate optimizer and code generator for every (language, target) *pair* — the classic "M languages $\times$ N targets" problem, needing on the order of $M \times N$ components instead of $M + N$.

A Bit of History

This $M \times N$ argument predates LLVM by decades. In the 1960s, **T-diagrams** (popularized by F. L. Bauer and others) were a standard notation for exactly this problem: a T-shaped diagram showing a compiler written in language H, translating source language S into target language T. Compiler writers used T-diagrams to reason about *bootstrapping* (compiling a new compiler using an existing one) and about how many compilers you actually need to support M languages on N machines — precisely the front-end/back-end decoupling argument above, expressed sixty years earlier with pen-and-paper diagrams instead of shared IR software.

Extra Depth: The Front/Middle/Back Split in a Real Toolchain — LLVM

The clearest concrete instance of this architecture today is LLVM, and it is worth walking through once here since we will return to it in Lecture 25 for code generation proper.

- **Front-end (Clang, for C/C++/Objective-C):** lexes and parses source into a Clang-

specific Abstract Syntax Tree (AST), performs semantic analysis and type checking on that AST, then *lowers* it into **LLVM IR** — a typed, SSA-based (Static Single Assignment) intermediate representation that is completely independent of C++ as a language. Rust (`rustc`), Swift, Julia, and Zig all have their own, completely different front-ends, but they all emit the *same* LLVM IR.

- **Middle-end (the LLVM optimizer, `opt`):** a pipeline of dozens of independent **passes** — e.g. `mem2reg` (promote stack variables to SSA registers), `instcombine` (peephole-simplify instruction sequences), `inline` (function inlining), `gvn` (global value numbering, subsuming common subexpression elimination), `loop-vectorize` — each of which reads LLVM IR and produces improved LLVM IR. None of these passes know or care whether the original source was C++, Rust, or Swift.
- **Back-end (`llc`, or the integrated code generator):** lowers optimized LLVM IR to a specific target's machine code — x86-64, AArch64, RISC-V, or even WebAssembly, selected via a `-target` triple. Instruction selection, register allocation, and instruction scheduling all happen here, specific to that one target.

The practical payoff: when Apple Silicon (AArch64) needed a good C++ compiler, no one had to write a new C++ front-end — Clang's C++ front-end and every middle-end optimization pass were already reusable; only a back-end for AArch64 was needed (and it mostly already existed, since LLVM had supported ARM for years). This is exactly the $M + N$ payoff from the coreidea box above, made concrete: roughly 20+ front-ends (Clang, `rustc`, Swift, Julia, ...) and roughly 20+ back-ends (x86-64, AArch64, RISC-V, WebAssembly, ...) share *one* middle-end, instead of each of those 20×20 combinations needing its own hand-written optimizer.

Extra Depth: The Front-End's Second Career — IDEs and Language Servers

The front-end (lexer + parser + semantic analyzer) is valuable even when no target code is ever generated. This observation underlies the **Language Server Protocol (LSP)**: an editor like VS Code or Neovim talks to a background process (a "language server") that re-runs a language's front-end on every keystroke to provide autocomplete, jump-to-definition, and real-time error squiggles — all without ever reaching code generation. `clangd` (built directly on Clang's front-end), `rust-analyzer`, and `gopls` are all, in effect, "compilers with the back end removed and an editor bolted on instead." This is a direct, practical payoff of the front-end/back-end separation described above: a front-end built for compilation can be repurposed wholesale for tooling.

2.4 Comparing Real-World IRs: LLVM IR, JVM Bytecode, and WebAssembly

[beyond DB] LLVM IR, JVM bytecode, and WebAssembly are all, loosely, "intermediate, machine-independent instruction sets sitting between source code and a real CPU" — which is why students often lump them together as interchangeable "bytecode." They are not interchangeable: each was designed with a different primary goal in mind, and that goal shapes how high- or low-level it is, what it assumes about memory and objects, and what it is actually good for.

	LLVM IR	**JVM Bytecode**	**WebAssembly (Wasm)**
Primary design goal	A reusable *compiler middle-end*: one optimizer, shared by many front-ends and back-ends	*Portability* ("write once, run anywhere") for Java-family, object-oriented, garbage-collected languages	*Safe, sandboxed* execution of code from an untrusted source (originally: the web), at near-native speed
Abstraction level	Low — close to assembly, but target-independent; typed and SSA-based	Medium-to-high — bakes in classes, interfaces, virtual method dispatch, exception tables	Low — a compact, portable, stack-based instruction set, close to assembly
Assumes an object model / GC?	No. No built-in notion of classes or garbage collection; each front-end supplies its own	Yes. Assumes the JVM supplies garbage collection and an object model	No, in Wasm's core spec. A separate, later "GC proposal" adds optional managed references
Control flow	Ordinary basic blocks and branches, the same shape as real machine code	Ordinary jumps/branches; the bytecode *verifier* checks type-safety, not control-flow shape	**Structured only** — blocks, loops, ifs; no arbitrary `goto`, so a validator can check it in a single linear pass
Usually executed directly?	No — almost always compiled further, to native code; not normally shipped as the final artifact users run	Yes — interpreted and/or JIT-compiled directly by a JVM	Yes — interpreted and/or JIT/AOT-compiled directly by a Wasm runtime
Best fit for	Being a shared layer that *very different* source languages (systems, functional, managed, ...) can all lower into	Languages that already fit the JVM's object/GC model well (Kotlin, Scala, Clojure, Groovy)	Any source language willing to compile to a small, safe, sandboxed core – often reached *via* LLVM's own WebAssembly back-end

Why LLVM IR's Low Level Is a Feature, Not a Limitation

LLVM IR deliberately does *not* assume any particular memory-management strategy or object model. That is exactly why it can serve as the shared middle layer for languages with wildly different runtime models: C's manual memory management, Rust's ownership/borrow-checked memory, Swift's automatic reference counting, and Julia's dynamic dispatch all lower into the same LLVM IR, because none of those choices had to be baked into the IR itself – each front-end encodes its own memory-management decisions *as ordinary LLVM IR instructions*. JVM bytecode makes the opposite trade-off: by assuming garbage collection and an object model up front, it becomes an excellent, low-friction target for languages that already want exactly that (Kotlin, Scala) but an awkward one for a language like C that manages memory itself. Neither design is "better" in general – they optimize for different things, which is precisely the point of this comparison.

Example: WebAssembly as "Just Another LLVM Back-End"

Because LLVM already has back-ends for x86-64, AArch64, and RISC-V, adding a WebAssembly back-end fits the exact same $M + N$ story from earlier in this lecture: any existing LLVM front-end (Clang for C/C++, `rustc` for Rust) can already produce WebAssembly simply by selecting the `wasm32` target, with *no* changes to the front-end or to any optimization pass. This is why, in practice, a great deal of WebAssembly in the wild is not hand-written but is the output of compiling C, C++, or Rust source through ordinary LLVM tooling (e.g. Emscripten for C/C++) – WebAssembly is simultaneously a stand-alone IR in its own right *and* one more entry in LLVM's list of back-end targets.

Exam Focus

Do not confuse the two axes this section and the earlier "kinds of IR" discussion test separately: *how low-level is it, and why* (this section) versus *when is it executed relative to run time* (Lecture 1's AOT/interpreter/JIT distinction). LLVM IR and WebAssembly are both low-level, but for different reasons – LLVM IR is low-level so that generating real machine code from it is easy and so that no language's runtime assumptions get baked in; WebAssembly is low-level *and* structured so that it can be validated quickly and executed safely in a sandbox. JVM bytecode is higher-level than both, by design, because portability for one specific family of managed languages – not raw compiler-infrastructure reuse, and not sandboxed safety for arbitrary source languages – was its original goal.

2.5 MLIR: A Framework for Building Compiler IRs

MLIR ("Multi-Level Intermediate Representation") is a compiler-infrastructure project started at Google around 2019 and merged into the official LLVM monorepo as one of its sub-projects in 2020, first described publicly in [MLIR]. It is worth understanding alongside LLVM IR for the same reason the previous section compared LLVM IR, JVM bytecode, and WebAssembly: people often assume "MLIR" is just another name for "LLVM IR, but newer," when in fact the two solve different problems and sit at different points in a compiler's pipeline.

2.5.1 The Problem MLIR Was Built to Solve

LLVM IR is deliberately low-level: typed, in SSA form, organized into ordinary basic blocks – close to assembly, as §2.4 described. That is exactly what makes it a good shared target for many different *source* languages once they have already been reduced to simple operations and control flow. It is a poor fit, however, for representing a program *before* that reduction has happened – for example, a matrix multiplication expressed as a single high-level tensor operation, or a nested loop that a compiler still intends to tile, fuse, or vectorize as a unit. Lowering straight to LLVM IR forces that structure to be flattened away immediately, before any optimization pass gets a chance to use it.

Before MLIR existed, projects that needed this kind of higher-level representation simply built their own IR from scratch: TensorFlow's XLA had its own IR for tensor computations, Swift had SIL, Rust had MIR, and so on. Each of these reimplemented the same underlying machinery – a verifier, a textual printer and parser, a pass manager, a way to track source locations for error messages – that LLVM already provides for LLVM IR, but for their own, unrelated IR. MLIR's goal is to give every such project a shared framework for this machinery, so that building a new, domain-specific IR is a matter of defining new operations rather than rebuilding compiler infrastructure from the ground up.

2.5.2 Core Ideas

Dialects. Instead of one fixed instruction set, MLIR defines an extensible mechanism for registering **dialects** – namespaced families of operations, types, and attributes. The `arith` dialect has operations like `arith.addi`; the `linalg` dialect has structured, high-level operations such as matrix multiplication; the `scf` dialect ("structured control flow") has `for` and `while` loops as single operations, rather than as jumps between basic blocks; and the `llvm` dialect defines operations that directly mirror LLVM IR. Crucially, operations from several different dialects can appear together in the very same function – there is no requirement to fully translate out of one dialect before using another.

Progressive lowering. A program does not jump from a high-level dialect straight to machine code in one step. It is lowered dialect by dialect: a high-level tensor operation becomes a structured loop, the structured loop becomes ordinary branching control flow, and that, in turn, becomes the `llvm` dialect. Each of these steps is a small, independently checkable transformation, and different compilers can choose to stop lowering at whichever level is most useful for a given optimization – a loop-tiling pass wants to run before loops are flattened into branches, for instance, while register allocation only makes sense long after that point.

Regions. An ordinary LLVM IR function is a flat sequence of basic blocks connected by branches – there is no notion of one block being "inside" another. An MLIR operation, by contrast, can carry a nested **region** containing its own blocks. A loop or a function body is therefore naturally represented as one operation containing a region, preserving its nested structure for as long as that structure remains useful, instead of being flattened into a control-flow graph immediately.

2.5.3 How MLIR Differs From LLVM IR

	LLVM IR	**MLIR**
What it is	One fixed intermediate representation	A framework for defining many intermediate representations (dialects)
Abstraction level	Low and fixed – close to assembly	Whatever a given dialect needs – high, low, or several levels mixed in one module
Control-flow structure	Flat basic blocks connected by branches	Operations can carry nested regions, so loops and other structure can stay nested until a pass chooses to flatten them
Role in a pipeline	The representation code is lowered *to*, immediately before LLVM's own optimizer and back-ends take over	Sits *upstream* of LLVM IR; its own `llvm` dialect is what eventually converts into ordinary LLVM IR

The last row is the point most often missed: MLIR does not replace LLVM. A program written using MLIR's higher-level dialects is progressively lowered, within MLIR, down to the `llvm` dialect; from there it is translated into ordinary LLVM IR and handed to the exact same optimizer passes and back-ends already described in §2.3 and §2.4. MLIR extends the *front* of the pipeline – giving compiler writers a place to do domain-specific reasoning (tensor shapes, loop tiling, hardware-specific fusion) before code is flattened down to LLVM IR – rather than replacing anything downstream of it.

2.5.4 Where MLIR Is Used

MLIR's dialect mechanism has made it attractive well beyond its original use case. Notable adopters include Google's XLA and IREE (compiling machine-learning computation graphs to CPUs, GPUs,

and custom accelerators), PyTorch's Torch-MLIR, Triton (a language for writing GPU kernels), CIRCT (an MLIR-based toolchain for hardware design, playing a role similar to parts of a Verilog toolchain), and Flang, LLVM's own Fortran front end, which lowers through an MLIR dialect called FIR before reaching LLVM IR (see [MLIR, MLIR-DOCS] for further reading).

2.6 Self-Check Questions

Practice – Not Graded

1. Place the following in correct pipeline order: semantic analysis, linking, assembly, lexical analysis, loading, syntax analysis, preprocessing, code generation.
2. A program calls a function defined in a library it never `#includes` a declaration for, but the linker still resolves the call successfully because the symbol name matches at link time. Which phase would have caught this as an error if the declaration truly were missing, and why didn't it?
3. Explain, in the M-languages/N-targets argument, why a shared IR turns an $M \times N$ problem into an $M + N$ problem. What is the "shared IR" in LLVM's case?
4. Why does dynamic linking trade smaller executables for slower startup, while static linking does the opposite?
5. `clangd` never performs code generation. Which phases of the pipeline does it still need to run, and why are those enough to power features like "jump to definition"?
6. For the `position` example in §2.1.1, write out what the three-address code would look like if `60` were replaced by `count`, an already-integer variable (so no `inttoreal` conversion is needed). Which of the six phases changes as a result, and which stay exactly the same?
7. JVM bytecode assumes garbage collection; LLVM IR does not. Explain why this makes JVM bytecode a natural fit for Kotlin but a poor natural fit for a language like C, and explain why WebAssembly's core spec makes the same choice as LLVM IR here rather than the JVM's.
8. For the statement `count = count + step`, list its lexemes, give the `<token-name, attribute-value>` pair for each one, and sketch the symbol table entries lexical analysis would create (assuming `count` and `step` are both integers).
9. Draw the syntax tree for `count = count + step` in the same style as the `position` example's tree, and identify which single node the semantic analyzer would need to annotate if `step` were declared `real` instead of integer.
10. For each of the following, classify the mistake as a lexical, syntax, or semantic error, and justify your answer in one sentence: (a) `int x = 5$;` (b) `if (x > 0 { y = 1; }` (missing a closing parenthesis) (c) calling `compute(x, y)` when `compute` was declared to take only one parameter.

Key Takeaways

- A compiler's six classical phases, in order: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation.
- These split into an **analysis** half (front end: builds an IR and symbol table) and a **synthesis** half (back end: builds the target program from that IR).
- The `position = initial + rate * 60` example shows all six phases acting on one tiny statement: tokens → syntax tree → annotated tree (type conversion inserted) → three-address code → optimized three-address code (constant folded) → target assembly.

- A **token** is an abstract `<name, attribute>` pair emitted for a recognized **lexeme** (the actual matched source text). A **symbol table** records one entry per declared identifier and is read from and written to by every phase – not a phase itself.
- **Lexical, syntax, and semantic errors** are caught by three different phases, in that order: a lexical error means no token pattern matches at all (e.g. an illegal character); a syntax error means every token is valid but their arrangement breaks the grammar (e.g. a missing semicolon); a semantic error means the program is grammatically fine but violates a meaning-level rule the language enforces (e.g. an undeclared identifier). A misspelled keyword is usually *not* a lexical error, since it is still a valid identifier lexeme.
- The **front-end/middle-end/back-end** split (language-specific $\rightarrow$ language-and-target-agnostic $\rightarrow$ target-specific) is what lets one shared IR (like LLVM IR) turn an $M \times N$ language/target problem into an $M + N$ problem.
- **LLVM IR, JVM bytecode, and WebAssembly** are all machine-independent instruction sets but were built for three different goals – reusable compiler infrastructure, portability for GC'd object-oriented languages, and safe sandboxed execution, respectively – and that goal explains how high- or low-level each one is and what it assumes about memory and objects.
- **MLIR** is not a fixed IR but a framework for defining many **dialects** at different abstraction levels, progressively lowered down to LLVM IR – it extends the front of the pipeline rather than replacing anything downstream of it.
- Front-ends have value beyond compilation: language servers (`clangd`, `rust-analyzer`) repurpose the same lexer/parser/semantic-analyzer pipeline for IDE tooling, without ever reaching code generation.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.

[MLIR] Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. arXiv:2002.11054, 2020.

[MLIR-DOCS] The MLIR Project. *Official Documentation*. https://mlir.llvm.org/

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 3

Lexical Analysis: Tokens, Patterns, and Lexemes; Regular Languages, Regular Expressions, and Regular Definitions

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.1 (The Role of the Lexical Analyzer), §3.3 (Specification of Tokens: terminology, strings, languages, operations, regular expressions, regular definitions, extensions)
Other references:	[AP] Modern Compiler Implementation, Ch. 2; [PP] Programming Language Pragmatics §3.3

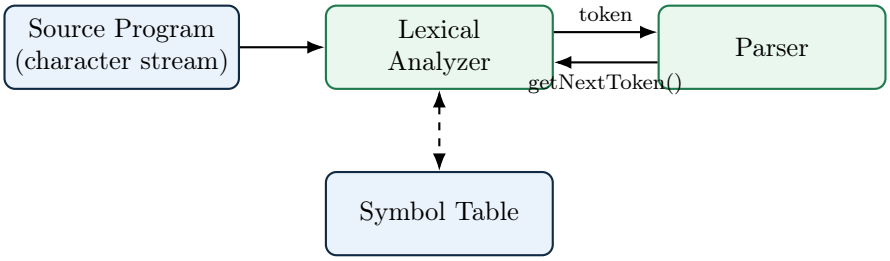
Lecture Roadmap

1. Where lexical analysis sits, and exactly how it talks to the parser; why lexer and parser are separate phases.
2. The three core vocabulary words of this unit: **token**, **pattern**, **lexeme** – with several fully worked examples, plus token **attributes** and **lexical errors**.
3. **The big picture**: how a regular expression eventually becomes a working, table-driven lexer, via finite automata – the roadmap for this lecture and the ones after it.
4. **Regular languages**, defined formally, and **regular expressions**, the notation we use to name them – the formal, inductive definition, plus many worked examples.
5. **Regular definitions**: naming sub-expressions, culminating in DB's classic identifier and numeric-literal definitions.
6. **Extensions** (`+`, `?`, character classes) – convenience, not new power – and **regular vs. non-regular languages**: what a lexer's patterns fundamentally cannot express.
7. How real-world lexers extend this model, and how the regular expressions in this lecture map directly onto the syntax used by **Flex**, the lexer-generator tool this course uses.

By the end of this lecture you will be able to read and write the formal notation a real lexer specification is built from – everything needed before we turn regular expressions into an actual running program via finite automata.

3.1 Where Lexical Analysis Fits

[DB] The **lexical analyzer** (also called a **scanner** or **lexer**) is the first phase of a compiler. It reads the raw stream of characters making up the source program and groups them into a stream of **tokens** — meaningful chunks like keywords, identifiers, numbers, and operators — that the rest of the compiler works with instead of raw text.



How the Lexer and Parser Actually Talk to Each Other

In nearly every real compiler, the lexer does *not* tokenize the whole file up front and hand the parser a list. Instead, the parser drives the process: whenever the parser needs another token, it calls a function conventionally named `getNextToken()`. The lexer resumes scanning exactly where it left off, produces the next single token, and returns it. This "pull" model, sometimes called treating the lexer as a **coroutine** of the parser, avoids ever materializing the full token list in memory and lets the lexer and parser interleave their work one token at a time.

3.1.1 Why Not Just One Phase?

[DB] DB gives three concrete engineering reasons for splitting lexical analysis out as its own phase rather than folding it into the parser:

1. **Simplicity of design.** Separating out low-level tasks (recognizing keywords, numbers, string literals) lets the parser work purely with grammatical structure, using tokens as its atomic symbols, instead of also worrying about individual characters.
2. **Improved efficiency.** Specialized, table-driven techniques for recognizing tokens – built from finite automata, which the lectures right after this one construct *directly* from the regular expressions we develop later in this very lecture – are considerably faster than the general-purpose parsing techniques would be if forced to operate character-by-character.
3. **Enhanced portability.** Input-device and operating-system-specific quirks — end-of-line conventions (`\n` vs. `\r\n`), character encodings, special end-of-file characters — are confined entirely to the lexer. The parser, and everything after it, is completely insulated from these differences.

Exam Focus

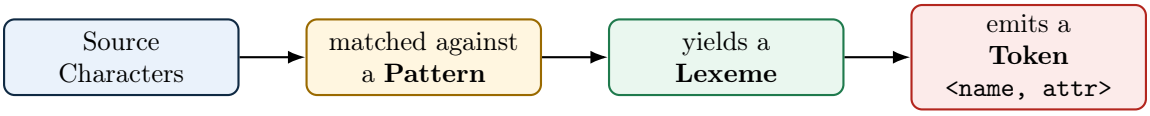
This "three reasons" list (simplicity, efficiency, portability) is a favorite short-answer question. Be able to name and briefly justify all three, not just one.

3.2 Tokens, Patterns, and Lexemes

[DB] These three terms are used precisely and are easy to blur together. Getting the distinction exactly right is one of the most exam-tested ideas in this entire unit.

Definition: Token, Pattern, Lexeme

- A **token** is a pair `<token-name, attribute-value>` representing a *category* of lexical unit, e.g. `ID`, `NUM`, `RELOP`. The token-name is an abstract symbol used by the parser.
- A **pattern** is the rule that describes the *set of possible lexemes* that can produce a given token – for identifiers, informally, "a letter followed by any number of letters and digits." §4.2, later in this lecture, makes patterns precise using regular expressions.
- A **lexeme** is the actual sequence of characters in the source program that *matches* a pattern and is classified as an instance of some token.



Example: DB-Style Token/Pattern/Lexeme Table

Token	Informal Description of Pattern	Sample Lexemes
`WHILE`	the exact keyword `while`	`while`
`ID`	a letter, followed by any number of letters or digits	`i`, `sum`, `score2`
`NUM`	any numeric constant	`0`, `42`, `3.14`
`RELOP`	<, <=, =, <>, >, or >=	<=, <>
`LITERAL`	any characters between a pair of double quotes, except a double quote itself	`"Total = "`

3.2.1 Worked Example 1: Tokenizing a Function Call

Consider the following line of C-like source code:

```
printf ( "Total = %d\n" , score ) ;
```

identifier punctuation/operator string literal

Lexeme	Token Name	Attribute-Value (passed to parser)
`printf`	`ID`	pointer to symbol-table entry for `printf`
`(`	`LPAREN`	none needed
`"Total = %d\n"`	`LITERAL`	pointer to the string's stored value
`,`	`COMMA`	none needed
`score`	`ID`	pointer to symbol-table entry for `score`
`)`	`RPAREN`	none needed
`;`	`SEMI`	none needed

3.3 Attributes for Tokens

[DB] When more than one lexeme can match a token's pattern (as with `ID` or `NUM`), the lexer must pass along enough extra information – the **attribute-value** – for later phases to know exactly *which* identifier or *which* number was matched. For identifiers, this is essentially always a pointer into the symbol table.

Example: Worked Example 2: An Assignment Statement

Source line: `E = M * C ** 2`
Assume `**` is this language's exponentiation operator. The lexer's output, as a stream of `<token-name, attribute-value>` pairs, is:

```
<ID, ptr to symtab entry for E>
<ASSIGN_OP, - >
<ID, ptr to symtab entry for M>
<MULT_OP, - >
<ID, ptr to symtab entry for C>
<EXP_OP, - >
<NUM, integer value 2>
```

Notice: `ID` tokens all carry a symbol-table pointer as their attribute so that later phases can distinguish `E` from `M` from `C`; the operator tokens need no attribute at all, since the token-name

alone (`ASSIGN_OP` vs. `MULT_OP`) already says everything the parser needs to know.

Exam Focus

Given a short program line, be able to produce the exact stream of `<token-name, attribute-value>` pairs, in order, the way both worked examples above do. This is the single most common long-answer question style for this unit.

3.3.1 Worked Example 3: A Complete Statement Block

```
int sum = 0;
while (i <= 10) {
sum = sum + i;
i = i + 1;
}
```

Lexeme	Token	Attribute	Lexeme	Token	Attribute
int	TYPE	–	sum	ID	ptr(sum)
sum	ID	ptr(sum)	=	ASSIGN	–
=	ASSIGN	–	sum	ID	ptr(sum)
0	NUM	0	+	PLUS	–
;	SEMI	–	i	ID	ptr(i)
while	WHILE	–	;	SEMI	–
(	LPAREN	–	i	ID	ptr(i)
i	ID	ptr(i)	=	ASSIGN	–
<=	RELOP	LE	i	ID	ptr(i)
10	NUM	10	+	PLUS	–
)	RPAREN	–	1	NUM	1
{	LBRACE	–	;	SEMI	–
			}	RBRACE	–

Two details worth noticing in this longer example:

- `sum` and `i` each get *their own* symbol-table pointer the first time they're seen, and the *same* pointer every time they reappear – the symbol table guarantees one entry per distinct identifier, no matter how many times it's used.
- Whitespace (the spaces and newlines between tokens) and indentation produce *no* tokens at all – they are consumed and discarded by the lexer, along with any comments, before the parser ever sees anything.

The Lexer's "Housekeeping" Duties

Beyond producing tokens, DB notes the lexer is also the natural place to: strip out whitespace and comments (neither produces a token); and track line numbers (and often column numbers) so that later phases can report errors with accurate source locations, e.g. "`undeclared variable 'x' at line 42`."

3.4 Lexical Errors

[DB] It is surprisingly rare for a lexer to be able to detect an error entirely on its own, *by itself, in isolation* – because almost any short character sequence is a *prefix* of some valid lexeme for some token.

Example: The Classic Illustration

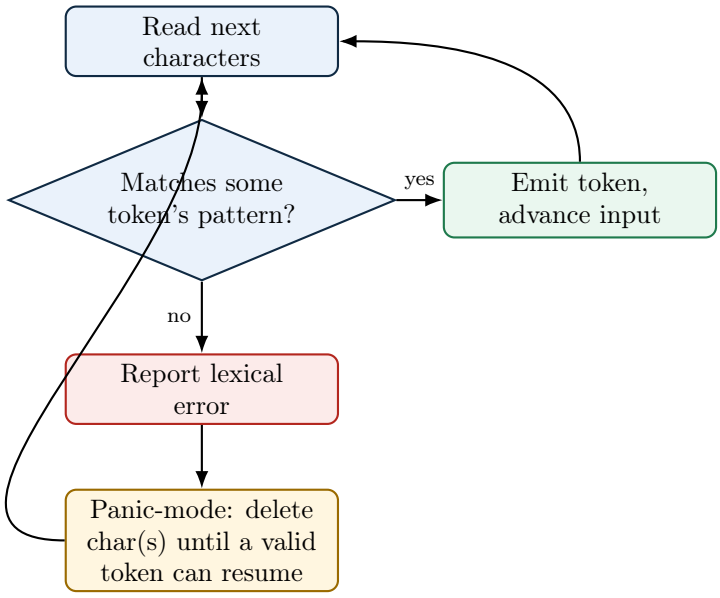
Consider the fragment `fi (a == f(x))` in a C-like language, where the programmer *meant* to type the keyword `if`. The lexer cannot know this. `fi` is a perfectly legal lexeme for token `ID` ("a letter followed by letters/digits"), so the lexer happily emits `<ID, ptr(fi)>`. The mistake will only be caught later – by the *parser*, if `fi` appears somewhere no identifier is grammatically valid, or by *semantic analysis*, if `fi` is used as if it were a function without ever having been declared.

Genuine lexical errors are ones where *no pattern for any token* can match the remaining input at all, or where a token's own internal structure is malformed:

- An illegal character that starts no valid token in the language, e.g. a stray `#` or `@` in a language that assigns those characters no meaning.
- An unterminated string literal: opening quote seen, but the line (or file) ends before a matching closing quote.
- A malformed numeric literal, e.g. `3.14.15` (two decimal points), or `2r` immediately followed by a letter with no valid meaning in that position.
- An unterminated comment: a `/*` seen with no matching `*/` anywhere before end of file.

3.4.1 Error Recovery Strategies

[DB] A production-quality lexer does not simply stop at the first error — it tries to recover and keep going, so the programmer can see *several* distinct errors from one compilation attempt rather than fixing them one at a time.



[DB] DB lists several concrete local-correction strategies a recovering lexer can attempt, beyond plain panic mode:

- **Deleting** an extraneous character.
- **Inserting** a missing character.
- **Replacing** one incorrect character with a correct one.
- **Transposing** two adjacent characters.

A Bit of History

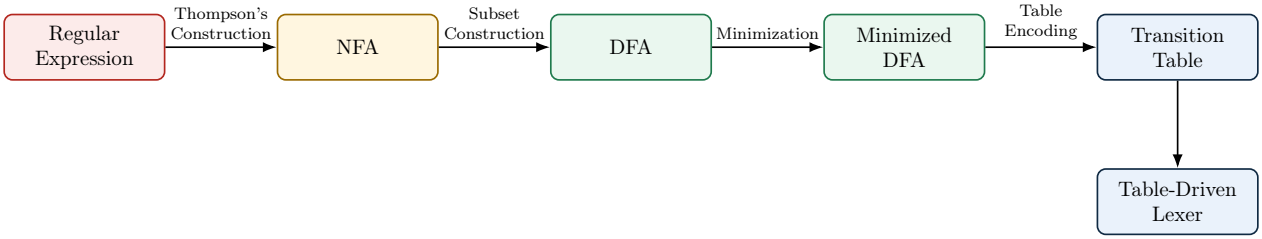
DB points toward a more principled version of this idea: choosing whichever single correction requires the *fewest* character edits – insertions, deletions, substitutions – to turn the bad input into something valid. This is exactly **edit distance** (Levenshtein distance), the same string-similarity measure used today in spell-checkers and DNA sequence alignment. Aho and Peterson described a minimum-distance error-correcting parser as early as 1972; the idea that "the best fix is the cheapest fix" turns out to generalize far beyond lexical analysis, into syntax error recovery and even runtime auto-correct systems.

Exam Focus

Be able to explain, with a concrete example like `fi (a == f(x))`, *why* a misspelled keyword is usually **not** a lexical error even though it looks like an obvious typo. Then be able to list at least two genuine, purely lexical errors (unterminated string, illegal character) and describe panic-mode recovery in one sentence.

3.5 The Big Picture: From Regular Expressions to a Working Lexer

[DB] Every pattern we have used so far has been informal English: "a letter followed by letters or digits." That is fine for talking about lexical analysis, but it is not something a computer can execute. Turning informal patterns into an actual running lexer takes a short pipeline of well-defined, *mechanical* steps, and it is worth seeing the whole pipeline once, up front, before building any one piece of it.



Step	What Happens
Regular expression → NFA	**Thompson's construction**: a purely mechanical algorithm that builds a nondeterministic finite automaton (NFA) directly from the structure of a regular expression, one small automaton per operator.
NFA → DFA	**Subset construction**: another mechanical algorithm that converts any NFA into an equivalent deterministic finite automaton (DFA) – one with no ambiguity about which transition to take, at the cost of (possibly) more states.
DFA → minimized DFA	**DFA minimization**: merges states that are behaviorally indistinguishable, producing the smallest possible DFA that still recognizes exactly the same language.
Minimized DFA → transition table	The (finite!) set of states and transitions is encoded as a 2-D table: `table[state][input_symbol] = next_state`.
Transition table → lexer	A tiny, fast loop – look up the current state and next character in the table, move to the next state, repeat – *is* the lexer's inner loop. No more cleverness is needed once the table exists.

Why This Pipeline Matters

Every single arrow in the diagram above is a *mechanical, automatable* algorithm – none of them require human judgment once the previous step is done. This is exactly why lexer-generator tools like Flex exist: you write only the left-most box (a set of regular expressions, one per token), and the tool runs the entire rest of the pipeline for you, emitting ready-to-compile scanner code. The remaining lectures on lexical analysis build this pipeline one arrow at a time – Thompson's construction and subset construction first, then minimization – so that by the time we reach Flex itself, you understand exactly what the tool is doing on your behalf rather than treating it as a black box.

3.6 Strings, Alphabets, and Languages

[DB] Before regular expressions can be defined precisely, three primitive terms need to be pinned down exactly as DB uses them.

Definition: Alphabet, String, Language

- An **alphabet** Σ is any finite set of symbols, e.g. $\{0, 1\}$ or the set of ASCII characters.
- A **string** (or *word*) over Σ is a finite sequence of symbols drawn from Σ. The **empty string**, written ε, has length 0.
- A **language** over Σ is simply *any* set of strings over Σ — possibly infinite, possibly empty ($\emptyset$, the language with no strings at all, which is *not* the same as $\{\varepsilon\}$, the language containing only the empty string).

Exam Focus

$\emptyset$ and $\{\varepsilon\}$ are a classic confusion. $\emptyset$ contains *zero* strings; $\{\varepsilon\}$ contains *one* string (the empty one). Keep them distinct.

3.6.1 Operations on Languages

[DB] Given two languages L and M over the same alphabet, DB defines four operations that build new languages out of old ones. These four operations are the entire foundation regular expressions – and, as the next section shows, the regular languages themselves – are built from.

Operation	Notation	Definition
Union	$L \cup M$	$\{ s \mid s \in L \text{ or } s \in M \}$
Concatenation	LM	$\{ st \mid s \in L \text{ and } t \in M \}$ (paste a string from L directly onto a string from M)
Kleene closure	L^*	$\bigcup_{i=0}^{\infty} L^i$ – zero or more strings from L, concatenated (includes ε, from L^0)
Positive closure	L^+	$\bigcup_{i=1}^{\infty} L^i$ – one or more strings from L, concatenated (excludes ε unless $\varepsilon \in L$)

Example: Worked Example: Letters and Digits

Let $L = \{a, b, c, \ldots, z, A, \ldots, Z\}$ (the 52 letters) and $D = \{0, 1, \ldots, 9\}$ (the 10 digits), each treated as a language of one-character strings.

- $L \cup D$ – every single letter *or* single digit: 62 strings, each of length 1.
- LD – every string formed by one letter followed by one digit, e.g. `a5`, `Z0` – exactly $52 \times 10 = 520$ strings, each of length 2.
- L^4 – every string of exactly 4 letters, e.g. `Test`, `abcd` – there are 52^4 such strings in total.
- L^* – every string of letters of *any* length, including ε — infinitely many strings.
- $L(L \cup D)^*$ – one letter, followed by zero or more letters-or-digits. This is *exactly* the language of identifiers in most C-like languages! We will turn this directly into a regular expression in §4.2.

3.7 Regular Languages: A Formal Definition

[DB] Everything is now in place to say precisely *which* languages qualify as "regular" – a statement about the language itself, as a set of strings, before we ever write a single regular expression on paper.

Definition: Regular Language (Inductive Definition)

The **regular languages** over an alphabet Σ are defined inductively, using exactly the union, concatenation, and Kleene-closure operations from §4.1.1:

- **Base cases:** $\emptyset$ is a regular language; $\{\varepsilon\}$ is a regular language; $\{a\}$ is a regular language for every symbol $a \in \Sigma$.
- **Inductive cases:** if L and M are regular languages, then so are $L \cup M$ (their union), LM (their concatenation), and L^* (the Kleene closure of L).
- **Nothing else** is a regular language – the regular languages are exactly the languages reachable by finitely many applications of the rules above, starting from the base cases.

This definition tells us *which* languages count as regular. It does *not*, by itself, give us a convenient way to write such a language down on paper or in code – and that gap is exactly what a regular expression fills.

Regular Language vs. Regular Expression

A **regular language** is a set of strings satisfying the definition above – a semantic object, a mathematical fact about which strings belong to it. A **regular expression** (defined next) is a piece of *syntax*: a compact, human-writable notation for naming one specific regular language. Every regular expression denotes exactly one regular language, and — a real theorem, proved properly in your Automata Theory course — every regular language can be named by at least one regular expression. The two notions therefore describe exactly the same class of languages, but they are conceptually different: one is a fact about a language, the other is a way of writing it down.

Exam Focus

"Is this a regular language?" and "is this a valid regular expression?" are different questions, and exam wording sometimes blurs them on purpose. A language can be regular even if you haven't yet written down a regular expression for it (you just haven't found the notation yet); conversely, every string of regex syntax that type-checks against the inductive definition in §4.2 denotes *some* regular language, whether or not that language is a useful one.

3.8 Regular Expressions

[DB] A **regular expression** is a compact notation for describing a language. DB defines regular expressions *inductively*: a small set of base cases, plus rules for combining smaller regular expressions into bigger ones – deliberately mirroring, symbol for symbol, the definition of the regular languages themselves from §3.7.

Definition: Regular Expressions (Inductive Definition)

Over alphabet Σ:

- **Base case 1:** ε is a regular expression; $L(\varepsilon) = \{\varepsilon\}$.
- **Base case 2:** for each symbol $a \in \Sigma$, a is a regular expression; $L(a) = \{a\}$.
- **Inductive case (union):** if r and s are regular expressions, so is $r \mid s$, with $L(r \mid s) = L(r) \cup L(s)$.
- **Inductive case (concatenation):** if r and s are regular expressions, so is rs, with $L(rs) = L(r)L(s)$.
- **Inductive case (closure):** if r is a regular expression, so is r^*, with $L(r^*) = (L(r))^*$.
- **Parenthesization:** if r is a regular expression, so is (r), with $L((r)) = L(r)$ – used purely to control grouping/precedence.

Here $L(r)$ denotes "the regular language named by regular expression r" – notice every base case and inductive case above produces a regular language in exactly the sense of §3.7, and nothing else, matching that definition rule for rule.

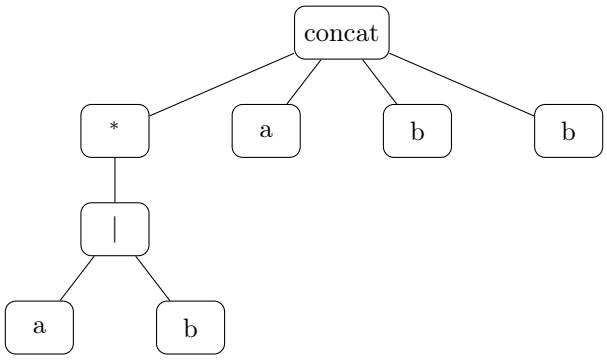
Operator Precedence

When parentheses are omitted, DB fixes a precedence, highest to lowest: **closure** (*) binds tightest, then **concatenation**, then **union** ($|$) binds loosest. So $ab \mid c$ means $(ab) \mid c$, *not* $a(b \mid c)$; and $a \mid bc^*$ means $a \mid (b(c^*))$.

3.8.1 Worked Example: Building Up a Regular Expression

DB's running example: the language of all strings of `a`'s and `b`'s that *end in* `abb`. Let's build the regular expression piece by piece.

Sub-expression	What it describes
$a \mid b$	a single `a` *or* a single `b`
$(a \mid b)^*$	*any* string of `a`'s and `b`'s (including ε)
$(a \mid b)^*abb$	any string of `a`'s and `b`'s, **followed by** the literal suffix `abb` – i.e. exactly the strings that *end in* `abb`



Syntax tree for $(a \mid b)^*abb$: the root concatenates four pieces — the starred union, then three literal symbols.

Example: Trace: Does `"babb"` Match?

`babb` = `b` (matched by the $(a|b)^*$ part, one repetition) followed by `a`, `b`, `b` (the literal suffix). Yes, it matches. Does `"abab"` match? No – it does not end in the literal substring `abb` (it ends in `bab`).

Exam Focus

Given an informal language description ("strings that start with …", "strings with an even number of …", "strings that don't contain …"), be able to construct the regular expression, and vice versa: given a regular expression, describe its language in English and list 2–3 example strings it does and does not match. This two-way translation is the most common exam pattern for this topic.

3.8.2 More Worked Examples

Language (in English)	Regular Expression	Matches / Doesn't Match
Binary strings with an even number of 0's	$1^*(01^*01^*)^*$	Matches `110011`; doesn't match `100`
Strings over $\{a, b\}$ containing at least one `a`	$(a \mid b)^*a(a \mid b)^*$	Matches `bba`; doesn't match `bbb`
Strings over $\{a, b\}$ *not* containing the substring `aa`	$b^*(ab^+)^*a^?$	Matches `ababb`; doesn't match `baab`
C-style single-line comment	`//`$(\Sigma \setminus \{\text{newline}\})^*$	Matches `// TODO fix this`; doesn't match a two-line comment

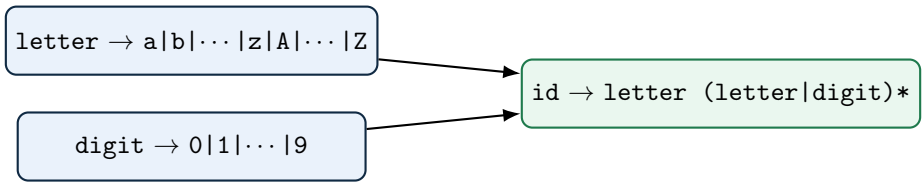
3.9 Regular Definitions

[DB] Writing out a complex regular expression as one giant unreadable formula is painful. A **regular definition** lets us name intermediate regular expressions and reuse those names in later definitions — exactly like defining helper variables in ordinary algebra.

Definition: Regular Definition

A regular definition is a sequence $d_1 \to r_1,\ d_2 \to r_2,\ \ldots,\ d_n \to r_n$ where each d_i is a new name, and each r_i is a regular expression over $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$ – that is, r_i may use the alphabet *and* any name defined *before* it, but never itself or a later name (no recursive or forward references).

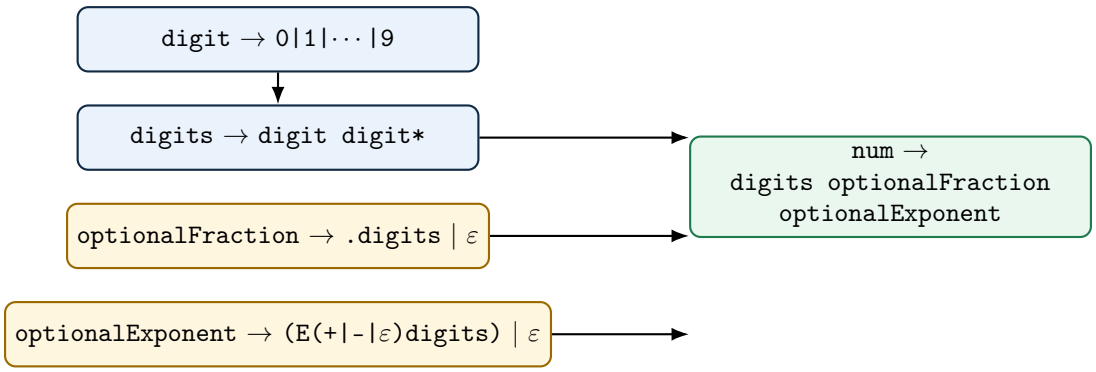
3.9.1 Worked Example: Identifiers



This is exactly $L(L \cup D)^*$ from the worked example in §4.1.1, now written as a named, reusable regular definition – and it is the pattern for token `ID` in almost every C-family language.

3.9.2 Worked Example: Unsigned Numbers (DB's Classic Definition)

Numbers like `5280`, `0.01`, `6.336E4`, or `1.89E-4` all need to be matched by a single token pattern. DB builds this up from four named pieces:



Example: Tracing `num` Against `6.336E4`

`digits` matches `6`; `optionalFraction` matches `.336` (the `.digits` branch); `optionalExponent` matches `E4` (the ε-sign branch of `(E(+|-|ε)digits)`, since no `+`/`-` appears). Concatenated: `6` + `.336` + `E4` = `6.336E4`. For a plain integer like `5280`, both `optionalFraction` and `optionalExponent` take their ε branch and contribute nothing.

Exam Focus

Be able to reproduce this exact four-piece `num` definition from memory, and trace it against a given number (integer, decimal, or scientific-notation) the way the example above does. This is DB's single most frequently examined regular-definition example.

3.10 Extensions of Regular Expressions

[beyond DB] Modern regex notation (and DB §3.3.5) adds a few extra operators. Formally, none of these add any expressive power beyond plain union/concatenation/closure — they are **syntactic**

sugar, purely for convenience and readability.

Extension	Meaning	Equivalent to Core Regex
r^+	one or more repetitions	rr^*
$r^?$	zero or one occurrence (optional)	$r \mid \varepsilon$
$[abc]$	character class: any one of `a`, `b`, `c`	$a \mid b \mid c$
$[a\text{-}z]$	character range: any letter from `a` to `z`	$a \mid b \mid \cdots \mid z$
$[\hat{}a]$	negated class: any symbol *except* `a`	union of all $x \in \Sigma \setminus \{a\}$

Exam Focus

Know that $r^+ \equiv rr^*$ and $r^? \equiv (r \mid \varepsilon)$ *exactly* — a common trick question asks you to rewrite an extended-notation regex using only the three core operators (union, concatenation, closure), which is precisely this substitution.

3.11 Regular vs. Non-Regular Languages

[DB] Not every language can be described by a regular expression. This matters enormously for compiler design: it is exactly *why* a single lexer pass is not enough, and why we need an entirely different, more powerful formalism (context-free grammars, covered once the course reaches parsing) for parsing.

The Central Limitation: No Counting

A regular expression (equivalently, a finite automaton, covered once the course reaches finite automata) has no way to *count* an unbounded quantity and later check that count against something else. It can verify local, bounded patterns, but it cannot verify *global balance*. This single limitation explains every classic example of a non-regular language.

Language	Regular?	Why
Identifiers: letter followed by letters/digits	**Yes**	Purely local: check each character against a fixed small set of rules, no counting needed.
Fixed-format numeric literals	**Yes**	Same reason – bounded structure, no unbounded counting or matching required.
Balanced parentheses, e.g. $\{a^n b^n \mid n \geq 0\}$	**No**	Requires counting how many `(` were seen and demanding *exactly* that many `)` later — an unbounded count with no fixed upper limit.
Matching HTML/XML open and close tags	**No**	Same reason, at the level of tags instead of characters – and worse, tags can *nest*, requiring a stack, not just a counter.
Palindromes over $\{a, b\}$	**No**	Requires comparing the first half of the string against the reverse of the second half – again, unbounded memory of what came before.

Example: Why Finite Automata Can't Count: A Concrete Illustration

Suppose you tried to build a finite automaton for balanced parentheses up to nesting depth n. You would need at least one distinct state per depth level, $0, 1, 2, \ldots, n$, just to remember "how many unmatched `(` have I seen so far." Since parentheses in real programs can nest arbitrarily deep, no *fixed, finite* number of states suffices for *every* possible program – yet a finite automaton is required to have a fixed number of states, decided in advance. This is the intuitive heart of the **pumping lemma for regular languages** from your Automata Theory prerequisite: pump the middle of a long enough matched string and you break the balance, proving no finite automaton can recognize it.

Bridging Idea – Why This Matters for the Rest of the Course

- Regular expressions/finite automata are the right tool for tokens: identifiers, numbers, keywords, operators – all locally-checkable, bounded patterns.
- Nested and recursive structure – matched parentheses, balanced braces, nested expressions, `if`/`else` matching – is fundamentally *beyond* what regular expressions can express, no matter how cleverly you write them.
- This is precisely why the course introduces **context-free grammars** once parsing begins: a strictly more powerful formalism (using a stack, conceptually) that *can* count and match nested structure, and precisely why the compiler needs *two* separate formalisms — one per phase — instead of one universal one.

3.12 Real-World Lexer Design

[beyond DB] Everything so far in this lecture follows DB's simplified model fairly closely. Production lexers extend that model in several practical directions worth knowing about, even though we will not build any of them by hand in this course.

3.12.1 Handling Keywords: The Reserved-Word Trick

A naive lexer might need a separate hand-written check for every keyword (`if`, `while`, `return`, ...) before falling back to the generic identifier pattern. Production lexers instead use a much simpler trick: pre-load the symbol table with every reserved word *before* scanning begins, each marked with its own keyword token. The lexer then has only *one* pattern for anything "letter followed by letters/digits." When it matches such a lexeme, it looks the lexeme up in the symbol table: if an entry already exists (because it's a reserved word), the lexer returns *that* token (e.g. `WHILE`); if no entry exists, it's a genuine user identifier, so the lexer creates a new symbol-table entry and returns `ID`. One pattern, one lookup, no special-casing in the automaton itself.

3.12.2 The Maximal Munch Rule

When several patterns could all match a prefix of the remaining input, real lexers apply the **longest-match** ("maximal munch") rule: always take the *longest* lexeme that matches *any* pattern. This is why `<=` is scanned as a single `RELOP` token and not as `<` followed by `=`, and why `i` in `int` is not prematurely cut off as an identifier before the lexer has looked far enough ahead to see the rest of the word. C's `--` (decrement) versus `- -` (two unary minuses) and `->` (member access through a pointer) are further classic examples where getting the lookahead right matters. We formalize *why* this rule is needed, and how a scanner implements it efficiently, once we cover finite automata in the lectures ahead.

3.12.3 Numeric and String Literals in Practice

DB's `NUM` pattern is deliberately simplified. Real languages support numeric literal formats far beyond plain decimal integers, and each variant is a distinct lexical pattern the scanner must recognize:

Format	Example
Hexadecimal / octal / binary	`0x1F`, `0o17`, `0b1010`
Scientific notation	`6.022e23`, `1.5E-10`
Digit-group separators	`1_000_000` (Python, Java, Rust, C++14)
Type suffixes	`3.14f` (float), `100L` (long), `42u` (unsigned)
Complex/imaginary literals	`3+4j` (Python)

Each of these is its own regular pattern (or family of patterns) that a real scanner's specification must include – one more reason lexer-generator tools like Flex (§3.13.5) are preferred over hand-written code once a language's literal syntax grows this rich.

String-literal lexing hides similar complexity. **Escape sequences** (`\n`, `\t`, `\\`, `\uXXXX`) must be recognized *inside* the literal's pattern, not treated as ending it. **Raw strings** (Python's `r"..."`, C#'s `@"..."`) deliberately *disable* escape processing, so a single string-literal token category can need two different sub-patterns. Hardest of all is **string interpolation / f-strings** (Python's `f"Total = {score}"`, JavaScript template literals): the lexer must recognize where an embedded *expression* begins inside a string and, in effect, briefly hand control back to a full expression lexer/parser before resuming the string literal. This breaks the assumption that lexical analysis is a single flat pass over characters, and is a genuinely active area of parser/lexer-design discussion in modern language implementations.

3.12.4 Beyond ASCII and Flat Text: Unicode and Indentation

DB's "a letter followed by letters and digits" quietly assumes ASCII. Modern language standards (Python 3, Swift, Julia, Rust as of recent editions) permit *any* Unicode character in the "Letter" category as part of an identifier – so accented Latin letters (`café`), non-Latin scripts (Greek, Arabic, Devanagari, Han characters), and even emoji are all valid identifiers in the languages that permit them (Swift famously allows an emoji as a variable name). Real lexer specifications reference the Unicode standard's character-category tables rather than a simple `[A-Za-z]` range.

Not every language is purely character-driven in the first place. Python and Haskell use indentation, not braces, to delimit blocks. Their lexers cannot rely on simple per-character regular patterns alone – they maintain an explicit **indentation stack** and synthesize special `INDENT` and `DEDENT` pseudo-tokens whenever a logical line's indentation increases or decreases relative to the stack's top, in addition to the ordinary tokens for the line's actual content. This is a clean example of *why* DB's model of "patterns matched against a flat character stream" is the right *starting* point but not the end of the story for every real language.

3.12.5 Seeing Real Tokenizers at Work

You do not have to take any of this on faith – most language toolchains expose their tokenizer directly. Python's `python3 -m tokenize myfile.py` prints every token, its exact source position, and its type. Clang's `clang -Xclang -dump-tokens -fsyntax-only file.c` dumps the full token stream Clang's own lexer produced. Node.js/Acorn and Babel offer their own `-tokens` flags for JavaScript. Running one of these on a file with a deliberate typo – an unterminated string, an unexpected character – is the fastest way to see exactly what a real lexer reports as an error versus what it silently accepts and passes on to the parser.

3.13 Regex in Practice

[beyond DB] The formal regular-expression notation from this lecture is the shared theoretical core behind every "regex" you have ever typed into a search box, a script, or a lexer specification – but real-world tools diverge from that core, and from each other, in important ways.

3.13.1 Not All "Regex" Is Actually Regular

This is a genuinely important and often-missed subtlety. Tools like Perl, Python's `re`, and JavaScript's regex support **backreferences** – e.g. `(\w+) \1` matches any word immediately repeated, like `hello hello`. This is provably *not* a regular language in the formal sense used in this lecture — recognizing it requires comparing two substrings for equality, which needs unbounded memory, just like the palindrome example in §4.5. Backreference-supporting engines use **backtracking search**, not finite automata, under the hood; they are more accurately called "backtracking pattern matchers" than true regular-expression engines. Lexer-generator tools like Flex, by contrast, implement only *true* regular expressions, compiled to genuine finite automata, precisely because lexical analysis needs the speed and predictability guarantees only real finite automata provide.

3.13.2 Catastrophic Backtracking (ReDoS)

Backtracking regex engines can, on adversarial input, take *exponential* time on a pattern that looks entirely innocent. The classic example is `(a+)+b` applied to a long string of `a`'s with no trailing `b`, like `"aaaaaaaaaaaaaaaaaaaaaaaaa!"`: the engine tries exponentially many ways of splitting the a's between the inner and outer `+` before giving up. This is a real, named security vulnerability class — **ReDoS** (Regular expression Denial of Service) — and has caused real production outages (Cloudflare's July 2019 global outage was triggered by exactly this kind of catastrophic backtracking in a WAF rule). The illustrative growth pattern:

Input length (a's, no trailing b)	20	25	30	35
Illustrative matching steps (order of magnitude)	$\sim 10^5$	$\sim 10^6$	$\sim 10^8$	$\sim 10^{10}$

This is exactly why DFA-based matching – which is *provably* linear in input length, with no possibility of this blow-up – is not just a theoretical nicety but the actual engineering reason production lexers never use a naive backtracking matcher.

3.13.3 Modern Guaranteed-Linear Engines

Google's **RE2** library and Rust's `regex` crate deliberately give up backreferences and a handful of other backtracking-only features specifically so that they can guarantee linear-time matching via automata-based execution, immune to ReDoS by construction. This is the industrial embodiment of the regular-vs-non-regular distinction from §4.5: by refusing to support genuinely non-regular features, these libraries buy back the performance guarantee that true finite automata provide.

3.13.4 Brzozowski Derivatives – An Alternative to Automata

The standard way to build a matcher for a regex is to first convert it to a finite automaton (via Thompson's construction, previewed in §3.5 and covered fully in the lectures ahead). There is a completely different, more recent technique worth knowing about: the **derivative** of a regular expression r with respect to a symbol a, written $\partial_a(r)$, is a new regular expression describing "what still needs to match after consuming an a." Repeatedly taking derivatives, one input symbol at a time, and checking whether the final derivative accepts ε, matches a string *without ever building an explicit automaton at all.* Janusz Brzozowski introduced this in 1964; it fell out of fashion for decades, then saw a resurgence starting around 2010 in functional-programming circles (Scala's and Racket's parser-combinator libraries use derivative-based matching), because derivatives compose beautifully with recursive data types in a way that hand-built automata do not.

3.13.5 Regular Expression Syntax in Flex

Flex, the lexer-generator tool this course uses, takes a set of regular expressions – one per token – and runs the *entire* pipeline from §3.5 for you, emitting a ready-to-compile C scanner. Its regex syntax is, deliberately, close to the formal core taught in this lecture, plus a handful of practical extensions:

Flex syntax	Meaning	This lecture's notation
`r1r2`	concatenation	$r_1 r_2$
`r1|r2`	union / alternation	$r_1 \mid r_2$
`r*`	Kleene closure	r^*
`r+`	one or more	r^+
`r?`	optional	$r^?$
`(r)`	grouping	(r)
`.`	any character except newline	shorthand for $\Sigma \setminus \{\text{newline}\}$
`[abc]`, `[a-z]`	character class / range	union of the listed symbols
`[^abc]`	negated class	union of $\Sigma \setminus \{a, b, c\}$
`{n}`, `{n,m}`, `{n,}`	exactly n (or n to m, or n or more) repetitions	r concatenated with itself, that many times
`"..."`	literal string – special characters inside need no escaping	concatenation of literal symbols
`\`	escape one special character	the literal symbol itself
`{name}`	reference a named regular definition from the Definitions section	exactly d_i in a regular definition (§4.3)
`^`, `$`	anchor: only at start / end of a line	a position assertion, not part of the core theory
`r1/r2`	trailing context: match r_1 only if followed by r_2 (lookahead, not consumed)	not part of the core theory

Example: The `num` Regular Definition, Written as Flex Would See It

§4.3.2's four-piece `num` definition translates almost verbatim into a Flex specification's Definitions section:

```
digit       [0-9]
digits      {digit}+
optFrac     (\.{digits})?
optExp      (E[+-]?{digits})?
num         {digits}{optFrac}{optExp}
```

Every named piece (`digit`, `digits`, `optFrac`, `optExp`, `num`) is exactly the d_i from the formal regular definition, just written with Flex's concrete `{name}` reference syntax instead of the abstract $d_i \to r_i$ notation – the underlying regular expression is identical either way.

Exam Focus

Two Flex features above – `{n,m}` interval counts and `r1/r2` trailing context – are *not* part of the six-rule inductive definition from §4.2. Be able to explain `{n,m}` as pure syntactic sugar (exactly like r^+ or $r^?$: it expands to n to m literal copies of r concatenated), the same way §4.4 treats the other extensions – but trailing context genuinely changes *which part of the match is consumed*, which is a new capability, not just notation.

A Bit of History

The regex you type into `grep`, Python, or a Flex specification are not identical languages – they share the core ideas from this lecture but differ in surface syntax and sometimes in power. POSIX ERE (`grep -E`) stays closest to DB's formal core, with no backreferences and character classes like `[:digit:]`. PCRE, Python's `re`, and JavaScript add backreferences and lookahead/lookbehind assertions – genuinely more expressive than regular languages, as §4.6 discussed. Knowing which flavor you're targeting matters: a pattern copied from a Python script into a Flex `.l` file may silently mean something different, or fail to compile at all.

3.14 Self-Check Questions

Practice – Not Graded

1. For the statement `total = total + rate * 60;`, write out the full stream of `<token-name, attribute-value>` pairs, following the style of Worked Examples 2 and 3.
2. Explain why `2x` (digit immediately followed by a letter) can be treated as a genuine lexical error in many languages, while `fi` cannot, even though both look like "obvious mistakes" to a human reader.
3. A lexer encounters an opening `"` with no matching closing quote before the end of the file. Describe two different, reasonable ways a lexer could recover from this and keep scanning the rest of the file.
4. Why must the longest-match (maximal munch) rule apply when scanning `i <= 10`, so that `<=` is not mistakenly split into `<` followed by `=`?
5. Explain, in your own words, why Python's lexer cannot be built from character-level patterns alone the way a C lexer can.

6. In the big-picture pipeline (§3.5), which single step is responsible for turning *ambiguity* (several possible next states) into a single, unambiguous next state? Name the algorithm.
7. Explain, without using the word "notation," the difference between a regular *language* and a regular *expression*.
8. Write a regular expression for all strings over $\{0, 1\}$ that *start* with `1`. Then write one for strings that *do not contain* the substring `00`.
9. Using DB's `num` regular definition from §4.3.2, trace the match for the literal `1.89E-4`, showing what each of `digits`, `optionalFraction`, and `optionalExponent` matches.
10. Rewrite r^+ and $r^?$ using only union, concatenation, and Kleene closure (no extension operators allowed).
11. Explain, in your own words and without using the word "count," why balanced parentheses of arbitrary depth cannot be recognized by any regular expression.
12. Translate the regular definition `id -> letter (letter|digit)*` into the `{name}`-based syntax a Flex Definitions section would use, matching §3.13.5's worked example.
13. A teammate writes the regex `(a|aa)+b` and complains their program hangs on long inputs with no trailing `b`. What phenomenon is this, and why does it happen with a backtracking engine but not with a DFA-based one?

Key Takeaways

- A **token** is an abstract `<name, attribute>` pair; a **pattern** is the rule describing which lexemes belong to a token; a **lexeme** is the actual matched source text. Keep these three cleanly separated.
- Lexer and parser are split into separate phases for **simplicity, efficiency, and portability** – know all three reasons.
- The lexer and parser typically interact via a pull-based `getNextToken()` call, not by materializing a full token list up front.
- Token attributes – almost always a symbol-table pointer for `ID` – are how later phases recover exactly *which* identifier or literal a token represents.
- Most apparent "typos" (like `fi` for `if`) are *not* lexical errors, because they are valid lexemes for some other token; genuine lexical errors are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers).
- The big picture: **regular expression → NFA → DFA → minimized DFA → transition table → lexer**, via Thompson's construction, subset construction, and minimization – every step mechanical, which is exactly why lexer generators can automate all of it.
- A **regular language** is a set of strings, defined inductively from $\emptyset$, $\{\varepsilon\}$, and $\{a\}$, closed under union, concatenation, and Kleene closure. A **regular expression** is the notation we use to name one – different concepts, same underlying class of languages.
- **Regular definitions** let you name and reuse sub-expressions; DB's `id` and `num` definitions are the two canonical worked examples to know cold.
- Extension operators (+, ?, character classes) add convenience, not expressive power – each has an exact equivalent using only the three core operators.
- Regular expressions fundamentally **cannot count** an unbounded quantity, which is exactly why balanced parentheses, nested tags, and palindromes are *not* regular – and exactly why the compiler needs a second, more powerful formalism (context-free grammars) for parsing.
- Not everything called "regex" in practice is regular in the formal sense – backreference-supporting engines use backtracking and can suffer catastrophic (ReDoS) performance; true

finite-automaton-based matching, which we build next, is immune to this by construction.

- Real-world lexers go well beyond DB's simplified patterns: the reserved-word trick, maximal munch, rich numeric/string literal formats, indentation-based tokens, and Unicode identifiers are all standard in production compilers – and Flex's own regex syntax (§3.13.5) maps almost directly onto everything formalized in this lecture.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 4

Regular Languages vs. Non-Regular Languages; Regular Expressions and Regular Definitions

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.3 (Specification of Tokens: Strings, Languages, Operations, Regular Expressions, Regular Definitions, Extensions)
Other references:	[PP] Programming Language Pragmatics §3.3

Lecture Roadmap

1. Strings, alphabets, and languages – the formal vocabulary underneath "pattern."
2. Operations on languages: union, concatenation, and closure.
3. **Regular expressions**: the formal, inductive definition, plus many worked examples.
4. **Regular definitions**: naming sub-expressions, culminating in DB's classic identifier and numeric-literal definitions.
5. Extensions (`+`, `?`, character classes) – convenience, not new power.
6. **Regular vs. non-regular languages**: what a lexer's patterns fundamentally cannot express, and why that is exactly the job handed to the parser.

Today makes "pattern" from Lecture 3 completely precise. Later lectures show *how* a machine matches a regular expression, via finite automata.

4.1 Strings, Alphabets, and Languages

[DB] Before regular expressions can be defined precisely, three primitive terms need to be pinned down exactly as DB uses them.

Definition: Alphabet, String, Language

- An **alphabet** Σ is any finite set of symbols, e.g. $\{0, 1\}$ or the set of ASCII characters.
- A **string** (or *word*) over Σ is a finite sequence of symbols drawn from Σ. The **empty string**, written ε, has length 0.
- A **language** over Σ is simply *any* set of strings over Σ — possibly infinite, possibly empty ($\emptyset$, the language with no strings at all, which is *not* the same as $\{\varepsilon\}$, the language containing only the empty string).

Exam Focus

$\emptyset$ and $\{\varepsilon\}$ are a classic confusion. $\emptyset$ contains *zero* strings; $\{\varepsilon\}$ contains *one* string (the empty one). Keep them distinct.

4.1.1 Operations on Languages

[DB] Given two languages L and M over the same alphabet, DB defines four operations that build new languages out of old ones. These four operations are the entire foundation regular expressions

are built from.

Operation	Notation	Definition
Union	$L \cup M$	$\{ s \mid s \in L \text{ or } s \in M \}$
Concatenation	LM	$\{ st \mid s \in L \text{ and } t \in M \}$ (paste a string from L directly onto a string from M)
Kleene closure	L^*	$\bigcup_{i=0}^{\infty} L^i$ – zero or more strings from L, concatenated (includes ε, from L^0)
Positive closure	L^+	$\bigcup_{i=1}^{\infty} L^i$ – one or more strings from L, concatenated (excludes ε unless $\varepsilon \in L$)

Example: Worked Example: Letters and Digits

Let $L = \{a, b, c, \ldots, z, A, \ldots, Z\}$ (the 52 letters) and $D = \{0, 1, \ldots, 9\}$ (the 10 digits), each treated as a language of one-character strings.

- $L \cup D$ – every single letter *or* single digit: 62 strings, each of length 1.
- LD – every string formed by one letter followed by one digit, e.g. `a5`, `Z0` – exactly $52 \times 10 = 520$ strings, each of length 2.
- L^4 – every string of exactly 4 letters, e.g. `Test`, `abcd` – there are 52^4 such strings in total.
- L^* – every string of letters of *any* length, including ε — infinitely many strings.
- $L(L \cup D)^*$ – one letter, followed by zero or more letters-or-digits. This is *exactly* the language of identifiers in most C-like languages! We will turn this directly into a regular expression in §4.2.

4.2 Regular Expressions

[DB] A **regular expression** is a compact notation for describing a language. DB defines regular expressions *inductively*: a small set of base cases, plus rules for combining smaller regular expressions into bigger ones.

Definition: Regular Expressions (Inductive Definition)

Over alphabet Σ:

- **Base case 1:** ε is a regular expression; $L(\varepsilon) = \{\varepsilon\}$.
- **Base case 2:** for each symbol $a \in \Sigma$, a is a regular expression; $L(a) = \{a\}$.
- **Inductive case (union):** if r and s are regular expressions, so is $r \mid s$, with $L(r \mid s) = L(r) \cup L(s)$.
- **Inductive case (concatenation):** if r and s are regular expressions, so is rs, with $L(rs) = L(r)L(s)$.
- **Inductive case (closure):** if r is a regular expression, so is r^*, with $L(r^*) = (L(r))^*$.
- **Parenthesization:** if r is a regular expression, so is (r), with $L((r)) = L(r)$ – used purely to control grouping/precedence.

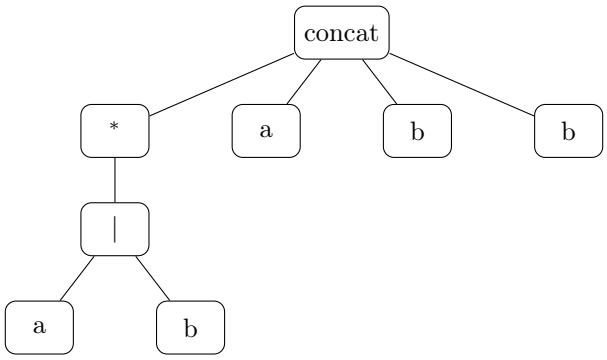
Operator Precedence

When parentheses are omitted, DB fixes a precedence, highest to lowest: **closure** (*) binds tightest, then **concatenation**, then **union** ($|$) binds loosest. So $ab \mid c$ means $(ab) \mid c$, *not* $a(b \mid c)$; and $a \mid bc^*$ means $a \mid (b(c^*))$.

4.2.1 Worked Example: Building Up a Regular Expression

DB's running example: the language of all strings of `a`'s and `b`'s that *end in* `abb`. Let's build the regular expression piece by piece.

Sub-expression	What it describes
$a \mid b$	a single `a` *or* a single `b`
$(a \mid b)^*$	*any* string of `a`'s and `b`'s (including ε)
$(a \mid b)^*abb$	any string of `a`'s and `b`'s, **followed by** the literal suffix `abb` – i.e. exactly the strings that *end in* `abb`



Syntax tree for $(a \mid b)^*abb$: the root concatenates four pieces — the starred union, then three literal symbols.

Example: Trace: Does `"babb"` Match?

`babb` = `b` (matched by the $(a|b)^*$ part, one repetition) followed by `a`, `b`, `b` (the literal suffix). Yes, it matches. Does `"abab"` match? No – it does not end in the literal substring `abb` (it ends in `bab`).

Exam Focus

Given an informal language description ("strings that start with …", "strings with an even number of …", "strings that don't contain …"), be able to construct the regular expression, and vice versa: given a regular expression, describe its language in English and list 2–3 example strings it does and does not match. This two-way translation is the most common exam pattern for this topic.

4.2.2 More Worked Examples

Language (in English)	Regular Expression	Matches / Doesn't Match
Binary strings with an even number of 0's	$1^*(01^*01^*)^*$	Matches `110011`; doesn't match `100`
Strings over $\{a, b\}$ containing at least one `a`	$(a \mid b)^*a(a \mid b)^*$	Matches `bba`; doesn't match `bbb`
Strings over $\{a, b\}$ *not* containing the substring `aa`	$b^*(ab^+)^*a^?$	Matches `ababb`; doesn't match `baab`
C-style single-line comment	$//(\Sigma \setminus \{\text{newline}\})^*$	Matches `// TODO fix this`; doesn't match a two-line comment

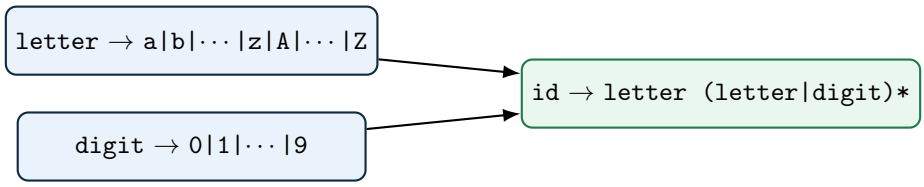
4.3 Regular Definitions

[DB] Writing out a complex regular expression as one giant unreadable formula is painful. A **regular definition** lets us name intermediate regular expressions and reuse those names in later definitions — exactly like defining helper variables in ordinary algebra.

Definition: Regular Definition

A regular definition is a sequence $d_1 \to r_1,\ d_2 \to r_2,\ \ldots,\ d_n \to r_n$ where each d_i is a new name, and each r_i is a regular expression over $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$ – that is, r_i may use the alphabet *and* any name defined *before* it, but never itself or a later name (no recursive or forward references).

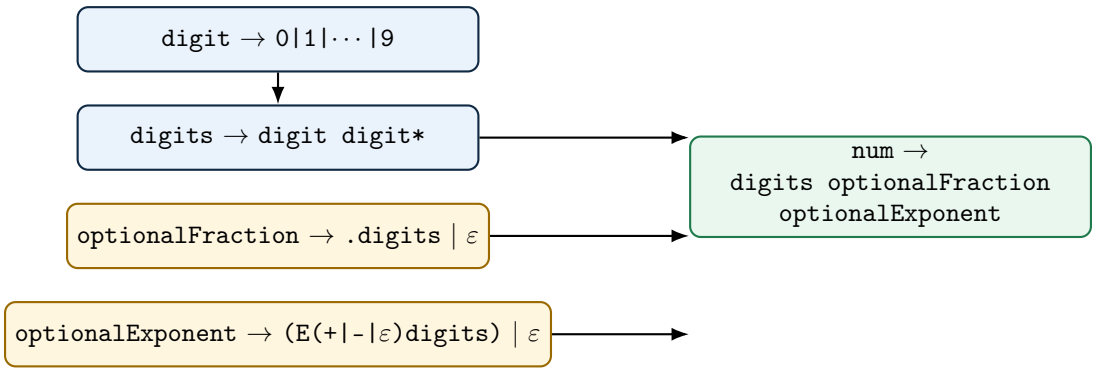
4.3.1 Worked Example: Identifiers



This is exactly $L(L \cup D)^*$ from the worked example in §4.1.1, now written as a named, reusable regular definition – and it is the pattern for token `ID` in almost every C-family language.

4.3.2 Worked Example: Unsigned Numbers (DB's Classic Definition)

Numbers like `5280`, `0.01`, `6.336E4`, or `1.89E-4` all need to be matched by a single token pattern. DB builds this up from four named pieces:



Example: Tracing `num` Against `6.336E4`

`digits` matches `6`; `optionalFraction` matches `.336` (the `.digits` branch); `optionalExponent` matches `E4` (the ε-sign branch of `(E(+|-|ε)digits)`, since no `+`/`-` appears). Concatenated: `6` + `.336` + `E4` = `6.336E4`. For a plain integer like `5280`, both `optionalFraction` and `optionalExponent` take their ε branch and contribute nothing.

Exam Focus

Be able to reproduce this exact four-piece `num` definition from memory, and trace it against a given number (integer, decimal, or scientific-notation) the way the example above does. This is DB's single most frequently examined regular-definition example.

4.4 Extensions of Regular Expressions

[beyond DB] Modern regex notation (and DB §3.3.5) adds a few extra operators. Formally, none of these add any expressive power beyond plain union/concatenation/closure — they are **syntactic**

sugar, purely for convenience and readability.

Extension	Meaning	Equivalent to Core Regex
r^+	one or more repetitions	rr^*
$r^?$	zero or one occurrence (optional)	$r \mid \varepsilon$
$[abc]$	character class: any one of `a`, `b`, `c`	$a \mid b \mid c$
$[a\text{-}z]$	character range: any letter from `a` to `z`	$a \mid b \mid \cdots \mid z$
$[\hat{}a]$	negated class: any symbol *except* `a`	union of all $x \in \Sigma \setminus \{a\}$

Exam Focus

Know that $r^+ \equiv rr^*$ and $r^? \equiv (r \mid \varepsilon)$ *exactly* — a common trick question asks you to rewrite an extended-notation regex using only the three core operators (union, concatenation, closure), which is precisely this substitution.

4.5 Regular vs. Non-Regular Languages

[DB] Not every language can be described by a regular expression. This matters enormously for compiler design: it is exactly *why* a single lexer pass is not enough, and why we need an entirely different, more powerful formalism (context-free grammars, covered once the course reaches parsing) for parsing.

The Central Limitation: No Counting

A regular expression (equivalently, a finite automaton, covered once the course reaches finite automata) has no way to *count* an unbounded quantity and later check that count against something else. It can verify local, bounded patterns, but it cannot verify *global balance*. This single limitation explains every classic example of a non-regular language.

Language	Regular?	Why
Identifiers: letter followed by letters/digits	**Yes**	Purely local: check each character against a fixed small set of rules, no counting needed.
Fixed-format numeric literals	**Yes**	Same reason – bounded structure, no unbounded counting or matching required.
Balanced parentheses, e.g. $\{a^n b^n \mid n \geq 0\}$	**No**	Requires counting how many `(` were seen and demanding *exactly* that many `)` later — an unbounded count with no fixed upper limit.
Matching HTML/XML open and close tags	**No**	Same reason, at the level of tags instead of characters – and worse, tags can *nest*, requiring a stack, not just a counter.
Palindromes over $\{a, b\}$	**No**	Requires comparing the first half of the string against the reverse of the second half – again, unbounded memory of what came before.

Example: Why Finite Automata Can't Count: A Concrete Illustration

Suppose you tried to build a finite automaton for balanced parentheses up to nesting depth n. You would need at least one distinct state per depth level, $0, 1, 2, \ldots, n$, just to remember "how many unmatched `(` have I seen so far." Since parentheses in real programs can nest arbitrarily deep, no *fixed, finite* number of states suffices for *every* possible program – yet a finite automaton is required to have a fixed number of states, decided in advance. This is the intuitive heart of the **pumping lemma for regular languages** from your Automata Theory prerequisite: pump the middle of a long enough matched string and you break the balance, proving no finite automaton can recognize it.

Bridging Idea – Why This Matters for the Rest of the Course

- Regular expressions/finite automata are the right tool for tokens: identifiers, numbers, keywords, operators – all locally-checkable, bounded patterns.
- Nested and recursive structure – matched parentheses, balanced braces, nested expressions, `if`/`else` matching – is fundamentally *beyond* what regular expressions can express, no matter how cleverly you write them.
- This is precisely why the course introduces **context-free grammars** once parsing begins: a strictly more powerful formalism (using a stack, conceptually) that *can* count and match nested structure, and precisely why the compiler needs *two* separate formalisms — one per phase — instead of one universal one.

4.6 Beyond the Dragon Book: Regex in Practice

Not All "Regex" Is Actually Regular

This is a genuinely important and often-missed subtlety. Tools like Perl, Python's `re`, and JavaScript's regex support **backreferences** – e.g. `(\w+) \1` matches any word immediately repeated, like `hello hello`. This is provably *not* a regular language in the formal CS-theory sense used in this lecture — recognizing it requires comparing two substrings for equality, which needs unbounded memory, just like the palindrome example above. Backreference-supporting engines use **backtracking search**, not finite automata, under the hood; they are more accurately called "backtracking pattern matchers" than true regular-expression engines. Meanwhile, lexer-generator tools like Flex implement only *true* regular expressions, compiled to genuine finite automata, precisely because lexical analysis needs the speed and predictability guarantees only real finite automata provide.

Catastrophic Backtracking (ReDoS)

Backtracking regex engines can, on adversarial input, take *exponential* time on a pattern that looks entirely innocent. The classic example is `(a+)+b` applied to a long string of `a`'s with no trailing `b`, like `"aaaaaaaaaaaaaaaaaaaaaaaaa!"`: the engine tries exponentially many ways of splitting the a's between the inner and outer `+` before giving up. This is a real, named security vulnerability class — **ReDoS** (Regular expression Denial of Service) — and has caused real production outages (Cloudflare's July 2019 global outage was triggered by exactly this kind of catastrophic backtracking in a WAF rule). The illustrative growth pattern:

Input length (a's, no trailing b)	20	25	30	35
Illustrative matching steps (order of magnitude)	$\sim 10^5$	$\sim 10^6$	$\sim 10^8$	$\sim 10^{10}$

This is exactly why DFA-based matching – which is *provably* linear in input length, with no possibility of this blow-up – is not just a theoretical nicety but the actual engineering reason production lexers never use a naive backtracking matcher.

Modern Guaranteed-Linear Engines

Google's **RE2** library and Rust's `regex` crate deliberately give up backreferences and a handful of other backtracking-only features specifically so that they can guarantee linear-time matching via automata-based execution, immune to ReDoS by construction. This is the industrial embodiment of the regular-vs-non-regular distinction from §4.5: by refusing to support genuinely non-regular features, these libraries buy back the performance guarantee that true finite automata provide.

Brzozowski Derivatives – An Alternative to Automata

The standard way to build a matcher for a regex is to first convert it to a finite automaton (via Thompson's construction), covered later in the course. There is a completely different, more recent technique worth knowing about: the **derivative** of a regular expression r with respect to a symbol a, written $\partial_a(r)$, is a new regular expression describing "what still needs to match after consuming an a." Repeatedly taking derivatives, one input symbol at a time,

and checking whether the final derivative accepts ε, matches a string *without ever building an explicit automaton at all*. Janusz Brzozowski introduced this in 1964; it fell out of fashion for decades, then saw a resurgence starting around 2010 in functional-programming circles (Scala's and Racket's parser-combinator libraries use derivative-based matching), because derivatives compose beautifully with recursive data types in a way that hand-built automata do not.

Real-World Regex Flavors Differ

The regex you type into `grep`, Python, or a Flex specification are not identical languages – they share the core ideas from this lecture but differ in surface syntax and sometimes in power:

Flavor	Notable differences
POSIX ERE (`grep -E`)	Closest to DB's formal core; no backreferences; character classes like `[:digit:]`.
PCRE / Python `re` / JavaScript	Add backreferences, lookahead/lookbehind assertions, non-greedy `*?`, named groups – genuinely more expressive than regular languages.
Flex / lex specifications	Core regular expressions plus start conditions and direct integration with symbol-table actions – built to compile to a real DFA.

Knowing which flavor you're targeting matters: a pattern copied from a Python script into a Flex `.l` file may silently mean something different, or fail to compile at all.

4.7 Self-Check Questions

Practice – Not Graded

1. Write a regular expression for all strings over $\{0, 1\}$ that *start* with `1`. Then write one for strings that *do not contain* the substring `00`.
2. Using DB's `num` regular definition from §4.3.2, trace the match for the literal `1.89E-4`, showing what each of `digits`, `optionalFraction`, and `optionalExponent` matches.
3. Rewrite r^+ and $r^?$ using only union, concatenation, and Kleene closure (no extension operators allowed).
4. Explain, in your own words and without using the word "count," why balanced parentheses of arbitrary depth cannot be recognized by any regular expression.
5. A teammate writes the regex `(a|aa)+b` and complains their program hangs on long inputs with no trailing `b`. What phenomenon is this, and why does it happen with a backtracking engine but not with a DFA-based one?

Key Takeaways

- A language is just a set of strings; regular expressions are one *notation* for describing certain languages, built inductively from ε, single symbols, union, concatenation, and Kleene closure.
- Precedence, highest to lowest: closure (*), then concatenation, then union (|).
- **Regular definitions** let you name and reuse sub-expressions; DB's `id` and `num` definitions are the two canonical worked examples to know cold.

- Extension operators (+, ?, character classes) add convenience, not expressive power – each has an exact equivalent using only the three core operators.
- Regular expressions fundamentally **cannot count** an unbounded quantity, which is exactly why balanced parentheses, nested tags, and palindromes are *not* regular – and exactly why the compiler needs a second, more powerful formalism (context-free grammars) for parsing.
- Not everything called "regex" in practice is regular in the formal sense – backreference-supporting engines use backtracking and can suffer catastrophic (ReDoS) performance; true finite-automaton-based matching, which we build next, is immune to this by construction.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.